Style your App with CSS (Version 8.6.1)

Advanced Level • 4.5 Hrs • 10 Modules • Version 8.6.1

This learning path will teach you the core concepts of CSS and how to apply them within Studio Pro to get the most out of the Atlas UI framework.

Module 1: Introduction

In this module, you'll get introduced to the learning goals and use case of this learning path.

Lecture 1.1: Welcome

Welcome to this learning path, where you will learn the core concepts of Cascading Style Sheet (CSS). You will learn how to apply them within Studio Pro and how to get the most out of the Atlas UI framework.

At the end of this learning path, you'll be able to:

1. know what tools are needed to edit the CSS within the Atlas UI Resources framework.
2. understand what role CSS plays within web applications.
3. know how the Atlas UI framework is organised and the underlying technology so you can more easily create modular applications.
4. know how to apply and best practices for using CSS within Studio Pro.
5. understand the elements within the box model and their implications.
6. know about the role of the display property and how to position elements where you want them on a page.
7. understand what it means to make an application responsive.
8. learn about flexbox and the possibilities it opens up.

Lecture 1.2: Audience & Duration

To follow this learning path, it is important that you have already followed the Become a Rapid Developer learning path, and you are familiar with navigating within Mendix Studio Pro. However, you don't need any other development knowledge to take this learning path.
It should take you approximately 4 hours and a half to finish all the theory and assignments of this learning path.

© Mendix Technology BV 2022. All rights reserved.

Lecture 1.3: Questions and Answers

Something not working?

- **Contact Mendix Community**

Mendix Forum is where you can find help when you get stuck or help others when you know the answer! You can first check whether someone has already asked the question you have. If not, feel free to post it and our community will help you out! After all, you gain points when you do so ;)

- **Submit a Support ticket**

If something really appears to be broken from a technical point of view (for example, you can't even visit or edit your app anymore), contact our Support Team, and they will do their absolute best to help you out!
\];
Got some feedback?

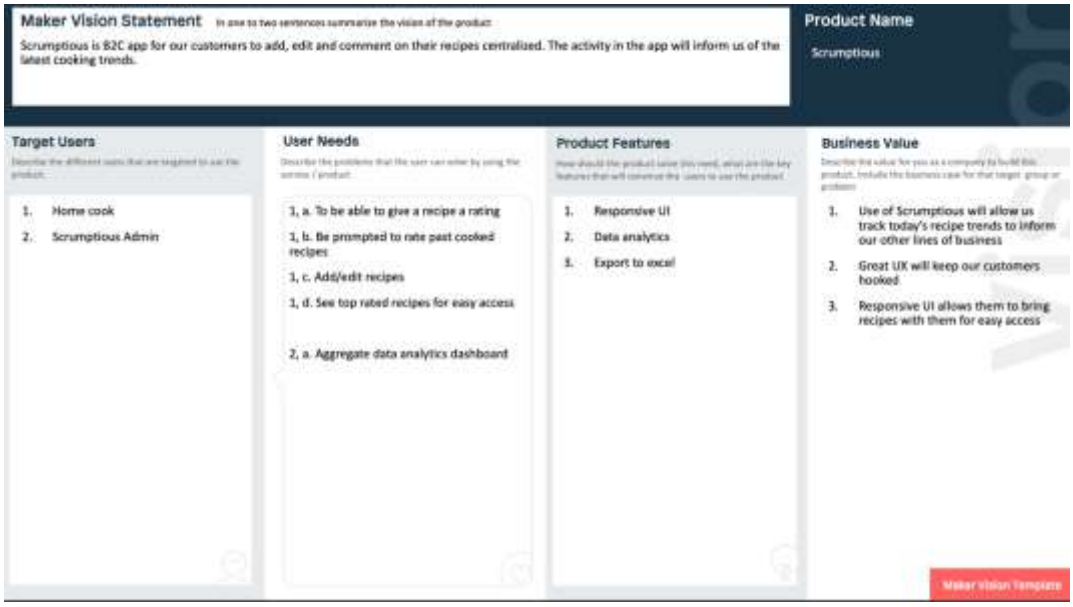
- **Fill out the Survey**

At the end of each learning path, you will be asked to fill in a survey. Please tell us what you think!

Lecture 1.4: Use Case

For this use case you will be working as the CSS developer for the recipe app 'Scrumptious'. You have an eye for detail, and love to find creative ways to make the front-end of the app as engaging as possible. Therefore, you took it upon yourself to use your CSS knowledge and enrich the front end of the app. It is a responsive app that allows you to add, edit, view and rate your recipes.

This is the vision board that outlines the objectives and functional needs of the application. Have a look before you get started!



© Mendix Technology BV 2022. All rights reserved.

Module 2: Setting Up your Project

In this module we will be install the necessary tools needed to style the Scrumptious and all mendix applications.

Lecture 2.1: Introduction

The main tools that you will use when styling your app are the following:

Visual Studio Code

Firebug/Chrome Inspector

Calypso, Sass Compiler

- Visual Studio Code (VSCode): There you can add and edit the Sass code
- Calypso: Compiles your Sass code into CSS ready for the the web browser (Chrome) to read it.
- Chrome: There you can see how your CSS looks on your web app

In the following lectures you will learn more about these tools, and how they can help you style your apps.

Lecture 2.1.1 : Download the Project

In this learning path, you will be provided with various exercises that you will need to do. That will help you gain the necessary practical experience. You will be doing all these exercises in a Mendix app. To speed up the process, we created the app that will be the basis on which you will build.

Lecture 2.1.2: Download VSCode

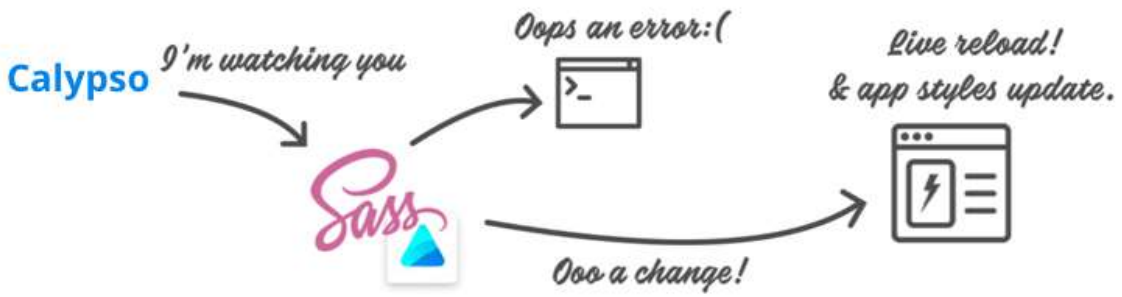
In order to style a Mendix application you will need a source-code editor such as VSCode. We recommend a source-code editor like VSCode because it allows you to open a whole project's folder which is key when jumping between files
You can **Download** VSCode here: https://code.visualstudio.com/.

Lecture 2.3: What is Calypso?

Calypso is known as a Sass compiler, which we will explain in greater detail in Module 4: Understanding Sass. It is the tool that watches the sass files for changes and then on saves converts the Sass files into CSS.

How it works:

© Mendix Technology BV 2022. All rights reserved.



1. Have you your project running locally.
2. Set up Calypso and choose your currently running app folder and click start. This allows it to watch the sass files for any changes to the files in the theme folder of your running app.
3. When you make a change to a sass file and notify Calypso, by saving the files you're working on, Calypso will first check to see if there are any semantic errors in your CSS code.
4. If it finds an error, Calypso will output an error log with the name of the file and line number it occurred on.
5. Once the error is fixed and saved Calypso will recompile the sass files into a Calypso file for the web app to read. But it doesn't stop there Calypso will auto reload your browser so can see the styling changes in real time.

Lecture 2.3.1: Download and Set Up Calypso

1. Download Calypso: https://www.dropbox.com/sh/46hip6pq8s9ipqp/AAAHBwm-8qcv9bQKSR7BrirCa?dl=0
2. Follow the instructions in the documentation to set it up https://docs.mendix.com/howto/front-end/Calypso#1-introduction

Having Calypso up and running is integral to doing styling on Mendix. If you don't have it working, then any styling changes you make in the framework will not be seen in your web application.

Lecture 2.4: Modern Browser, Chrome or Firefox

For this learning path we recommend you use a modern web browser, like Chrome. If you don't already have Chrome on your machine you can download it here: https://www.google.com/chrome/

© Mendix Technology BV 2022. All rights reserved.

Lecture 2.5: Tips and Tricks for Managing Application Windows

As you can see, you have to use multiple applications to do styling. I find the easiest way to develop is to have all the application windows open. Then, for mac users, use “command + tab” and windows use “alt + tab” to jump between applications.

Lecture 2.6: Summary

In this module, you learned about the tools required to do CSS development on Mendix. In addition, you downloaded the project and all the required tools so that you are ready to begin your journey as a CSS developer.

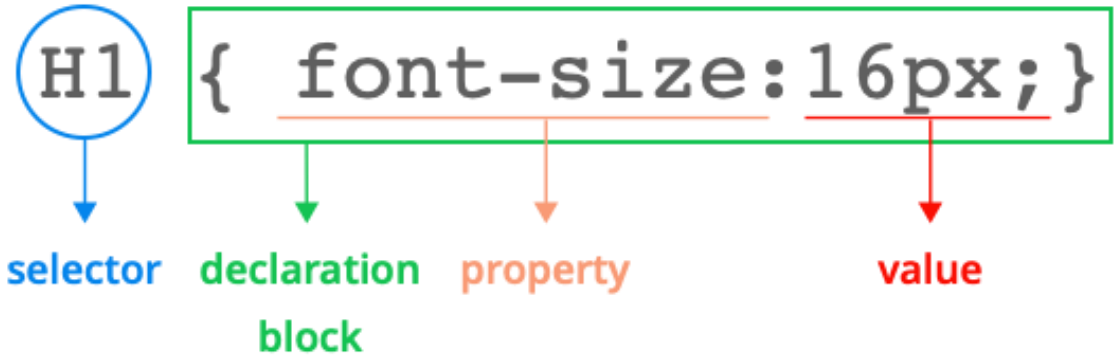
Module 3: Understanding CSS

In this module we will be learning about CSS and how it is read by the web browser in order to display the styling to the application user.

Lecture 3.1: What is CSS

CSS is a style sheet language used on web applications to display the content within a markup language like HTML. CSS presents to the users of websites and web apps the layouts, colors, fonts, and all other visualizations of content.
CSS stands for cascading style sheets.

Below is the syntax structure of a CSS declaration:



Lecture 3.2: What are Selectors?

In CSS, selectors are the connection between the CSS and the HTML. The selector is a reference to the object that you want to be styled in the HTML. Understanding how selectors work and how to make your selectors target the specific HTML object is crucial to making your application looking the way you want.

In Mendix the primary selectors used are **classes** and **html tags.**

© Mendix Technology BV 2022. All rights reserved.

```
.class-name
<h1></h1>
```

You can add as many classes to one selector as you want. Adding multiple classes to one selector is called chaining selectors.

Nested elements can be selected by separating selectors with a space. This is a great way to target a specific selector.

```
Ul li a
```

The example above targets a link tag within a li tag with a ul tag.

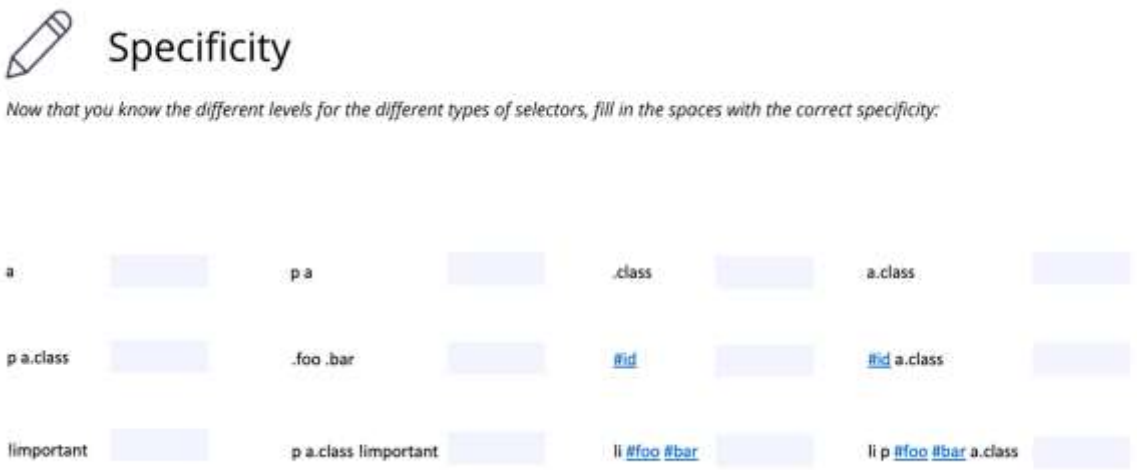
Lecture 3.3: What is Specificity

In CSS, there is a concept of specificity which defines how a browser decides which style declaration is displayed based on "weights" for each selector type.

Below illustrates the weight levels for the different types of selectors you will find in HTML

<p> (HTML tag)				**1**
.class (Class Selector)			**1**	0
#id (id Selector)		**1**	0	0
!important/ inline styles	**1**	0	0	0

You can use this template to fill in the specificity:



When your selector is made up of multiple classes and/or HTML tags, you add those points up, to get the weight of your selector. So if your selector has less weight than an existing selector, then your style declaration will not be shown.

As a general styling best practice, the use of !important or inline styling is generally frowned upon as it can make it harder to overwrite.

© Mendix Technology BV 2022. All rights reserved.

Lecture 3.4: Summary

In this module, you learned what CSS is and its role in web applications. You also learned about some key concepts like selectors and specificity that power CSS to make your applications look the way you envision it to.

Module 4: Understanding Sass

In this module we will be learning about SASS, which is the technology that makes up Mendix's UI framework. We will also be starting to modify Atlas to better match the UI design of the Scrumptious app.

Lecture 4.1: What is Sass

SASS is A CSS preprocessor, a coding language that is similar to CSS but adds advanced features that help achieve more reusable, maintainable, and extensible CSS code.

The Atlas UI framework uses Sass so that we at Mendix can provide a solution that works if you're creating a desktop app, mobile app, or anything in-between. It affords us a level of organization that, in plain CSS, would be unwieldy and cumbersome to maintain.

Lecture 4.2: Key Sass Features

Sass allows us to modularize CSS code into digestible chunks, aka partials. Other features available in Sass include:

- variables, for quick reference for comonly used and hard to remember values,
- nesting, allows you to mirror the HTML markup structure
- functions, aka mixins, lets you make groups of CSS declarations that you want to reuse throughout your site. You can even pass in values to make your mixin more flexible.

Lecture 4.3: File types and @import

There are two distinct file naming conventions when dealing with Sass.

- Partials are Sass files that represent the modularized chunks of code
- Partials always start with an underscore in the naming convention
- Files that you want converted to CSS don't have an underscore in the name

Sass builds upon the CSS concept of @import

- With CSS, every @import requires an HTTP request, which will inevitably slow down your page loading time.
- @import in Sass acts as the glue that binds all your partials together

© Mendix Technology BV 2022. All rights reserved.

Files with the word variable in the name

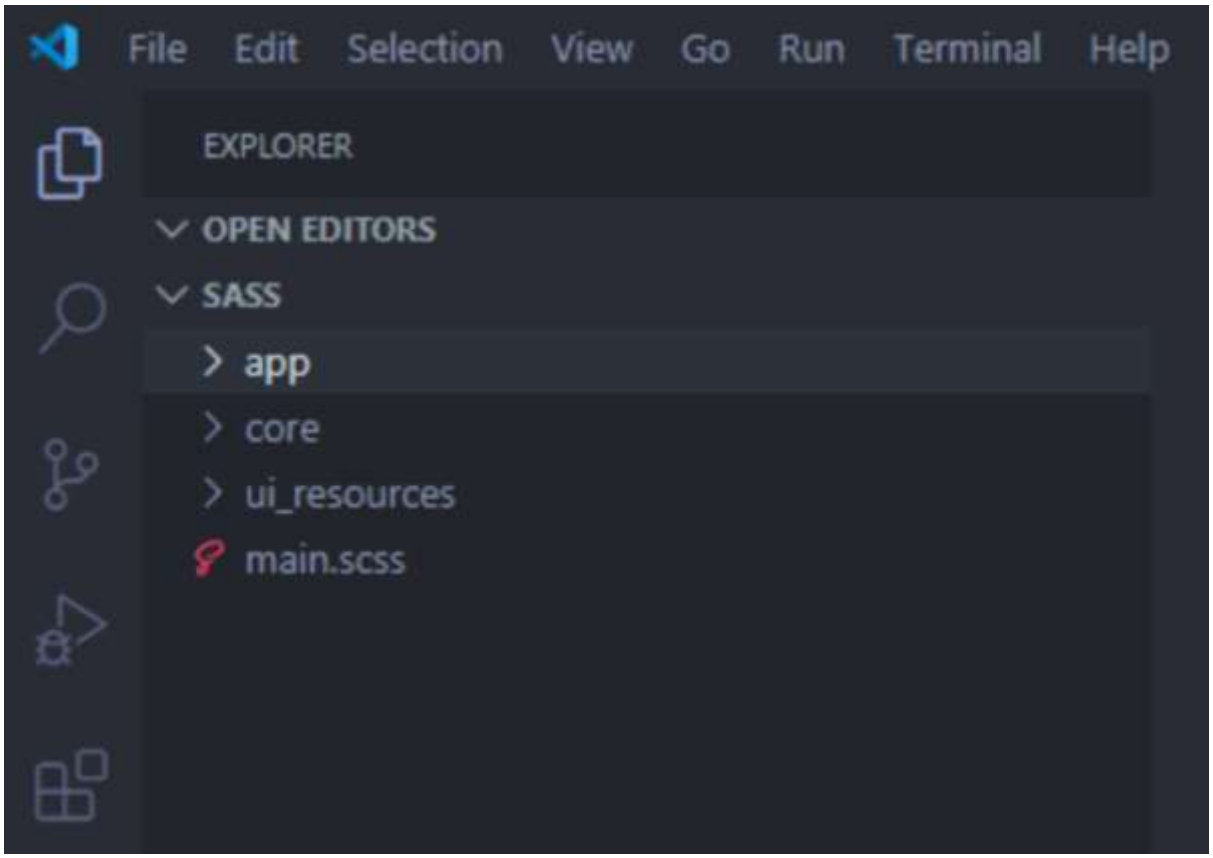
- Although not a strict rule in an effort to keep the code organized it is best to keep only $variables in partials labeled with the name variable and no style declarations or @import

Lecture 4.3.1: Create a Partial

The goal of this first exercise is to create a partial where you can add styles that are specific to your project. You will create a partial called scrumptious and check to make sure it is integrated into our Sass framework.

1. Open the project **StylingLearningPath**.
2. Click **Run Locally**.
3. Open up Calypso. Make sure that the App folder selected is the Styling Learning Path project and click **Start**.
4. Open up Visual Studio Code and click navigation item **File > Open Folder…**
5. An explorer window should appear. Navigate to your project's folder and click through to **theme > styles > web > sass** and click **Open**.

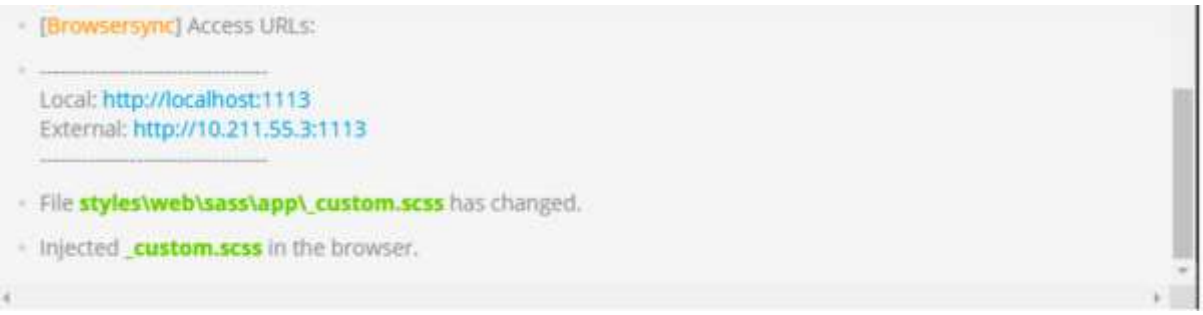
Note: When doing CSS development with Mendix, steps 1-5 will become very familiar. These are the preparation steps you need to follow every-time you want to apply styling to a project.



6. On the left-hand side of Visual Studio Code, you should see a folder tree. Expand the **app** folder. This is where you will add your custom partial.

© Mendix Technology BV 2022. All rights reserved.

7. With the app folder selected right-click and select **New File**.
8. Type in "**_scrumptious.scss**" and hit **Enter**.
9. Click the **_custom.scss** file. This file contains only @import declarations, which ensures that the Calypso included in each of these files is included in the outputted CSS file.
10. At the bottom of the file type: **@import "scrumptious"**; and hit **ctrl_S** (or **command _ S** for Mac)to save the changes to the file.
11. To make sure that the new partial was successfully added to the Calypso file switch back to Calypso and you should see a log message like this:



Congratulations you've added a partial. That partial is now empty, so you can't see any changes yet, but don't worry, you will get more chances to do so in the following exercises!

Lecture 4.3.2: Update the Sass Variable File

In this exercise, you are going to continue the work from exercise 4.3.1. You will be updating the colors and font sizes in the SASS variables file. At the start of any project, it is important to map the colors, fonts and font-sizes from the design files to the variables file. Setting up the variables file will make your future development much easier.

Note: make sure you have completed steps 1-5 of Exercise 4.1 so that these new changes will be incorporated into the compiled CSS.

1. In Visual Studio Code, navigate to the **_custom-variables** file under **sass > app** folder. Any file labeled with the word "variable" typically means it is reserved just to hold sass variables. To keep your sass organized you should not put any Calypso declarations in these partials.
2. Modify the brand color variables values and variables font-size-h1, font-size-h2, font-size-h3, font-size-h4 to match the below screenshot:

© Mendix Technology BV 2022. All rights reserved.

```
_custom-variables.scss ×
app > _custom-variables.scss > ...
//== Gray Shades
//## Different gray shades to be used for our variables and components
$gray-darker:                    #222222;
$gray-dark:                      #333333;
$gray:                           #555555;
$gray-light:                     #888888;
$gray-primary:                   #D7D7D7;
$gray-lighter:                   #EEEEEE;

//== Step 1: Brand Colors
$brand-default:                  #5D5D5D;
$brand-primary:                  #360738;
$brand-inverse:                  #211920;
$brand-info:                     #0094D7;
$brand-success:                  #76CA02;
$brand-warning:                  #FFFF29;
$brand-danger:                   #ED1C24;

// Used for other variables
$default-border-color:           $gray-primary;
$brand-primary-light:            #8414A0;

//== Step 3: Typography
$font-family-import:             "https://fonts.googleapis.com/css2?family=Montserrat:wght@400;700&display=swap";
$font-family-base:               "Montserrat", sans-serif;

$font-title-import:              "https://fonts.googleapis.com/css2?family=Playfair+Display:wght@700&display=swap";
$font-family-title:              "Playfair Display", serif;

$font-base-size:                 14px;
$font-base-color:                #555555;
$link-color:                     $brand-info;
$font-size-h1:                   40px;
$font-size-h2:                   30px;
$font-size-h3:                   24px;
$font-size-h4:                   20px;
$font-color-detail:              $gray-light;
$font-color-headers:             #173470;
```

In this variable file, the custom font variables have already been added with an import variable and declaration variable. You can learn more about importing fonts like we've done, here.

3. Save the variables file and switch over to the app running in the browser. You should see that the header button color has changed.

before after

Note: if you don't see changes in the browser, check to make sure that Calypso is running. If that doesn't work, check to see if you've saved the custom-variables file.

Congrats! You've modified the variables to better tailor your application to look like your brand.

© Mendix Technology BV 2022. All rights reserved.

Lecture 4.4: Summary

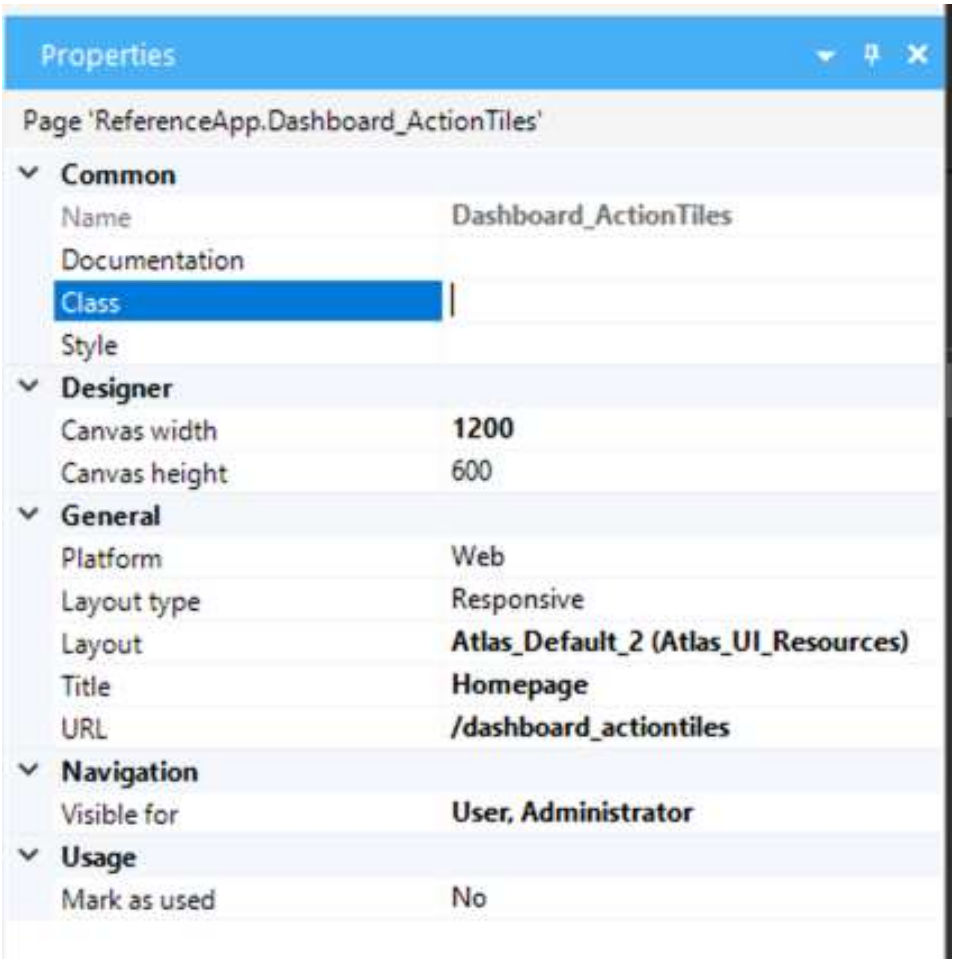
In this module, you learned what Sass is, its key features, and how it is leveraged within Atlas, Mendix's UI frameworks, to jump start the styling of your application.

Module 5: Applying styling

In the last module, you learned about SASS, the technology foundation of the Atlas UI framework. In this module, you will learn about the steps and tools needed to do CSS development with Mendix. You will also apply some simple Calypso rules to our application.

Lecture 5.1: Best Practice

Let's have a look at some best practices. In the screenshot below, you are looking at the properties of a dashboard widget. Which properties will you need to edit though?



Use the class field
Within the properties panel in Studio Pro there is a field called class.
You can access this by right-clicking and selecting view properties. The class field is found under the common tab.
In this field, you apply custom classes you want to be applied to your mendix objects, also known as widgets.

© Mendix Technology BV 2022. All rights reserved.

Don’t use the style field

Directly below the class field is one called style. Many novices front-end developers will use this to apply styling directly to a widget. As we become more experienced MX developers, it is viewed as best practice to use classes as opposed to the style field.

- Adding styles this way only lets you target the top-level item and is unable to reach any HTML objects nested within.
- Using the style field makes it hard to create a consistent look and feel because the developer has to remember every place they put an inline style for every page, which leads to mistakes.
- This inevitably creates a poor user experience through inconsistencies across pages.

Lecture 5.2: Development Workflow

There are roughly 4 steps you need to take to see a Calypso class take effect in the browser.

Note: You don't need to click along just yet. Just read and try to understand the steps. You will get plenty of opportunities to practice in the next lecture.

1. Create & Assign Custom CSS Class to the desired element in Studio Pro

2. Save and press run Run in Studio Pro. Using the web browser inspector find the custom class rendered in the HTML

3. Apply CSS properties in the browser inspector to get the object to look the way you want

© Mendix Technology BV 2022. All rights reserved.

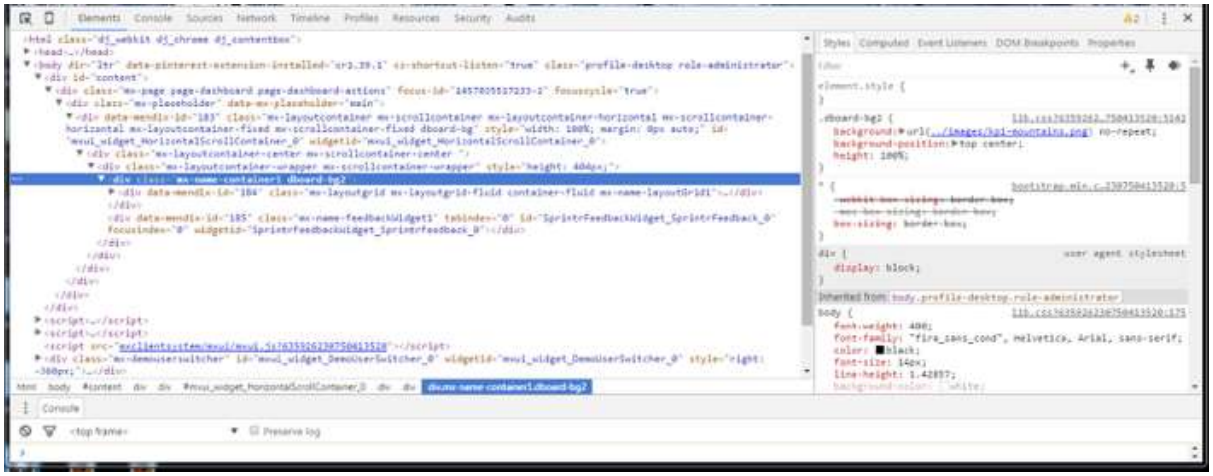
4. Copy styles from browser inspector, create a class from step 1 in sass file and paste styles between brackets.

```
.homepage-tiles {
    padding: 30px;
    border: none;
    border-radius: 12px;
    background-color: $brand-inverse;
    box-shadow: 0 5px 5px #121516;
```

Lecture 5.3: Web Browser Inspector

As the Web Browser Inspector was frequently mentioned in the previous lecture, let's spend a little time getting to know it. We recommend using modern browsers, such as Chrome and Firefox, to do most of your CSS development, because they come with fully equipped inspection tools. For this section, we will be talking specifically about Chrome's web inspection tool.

To open the web browser inspector, right-click on an object you want to inspect and select Inspect. There are 2 main sections for CSS development; Elements panel and CSS panel.

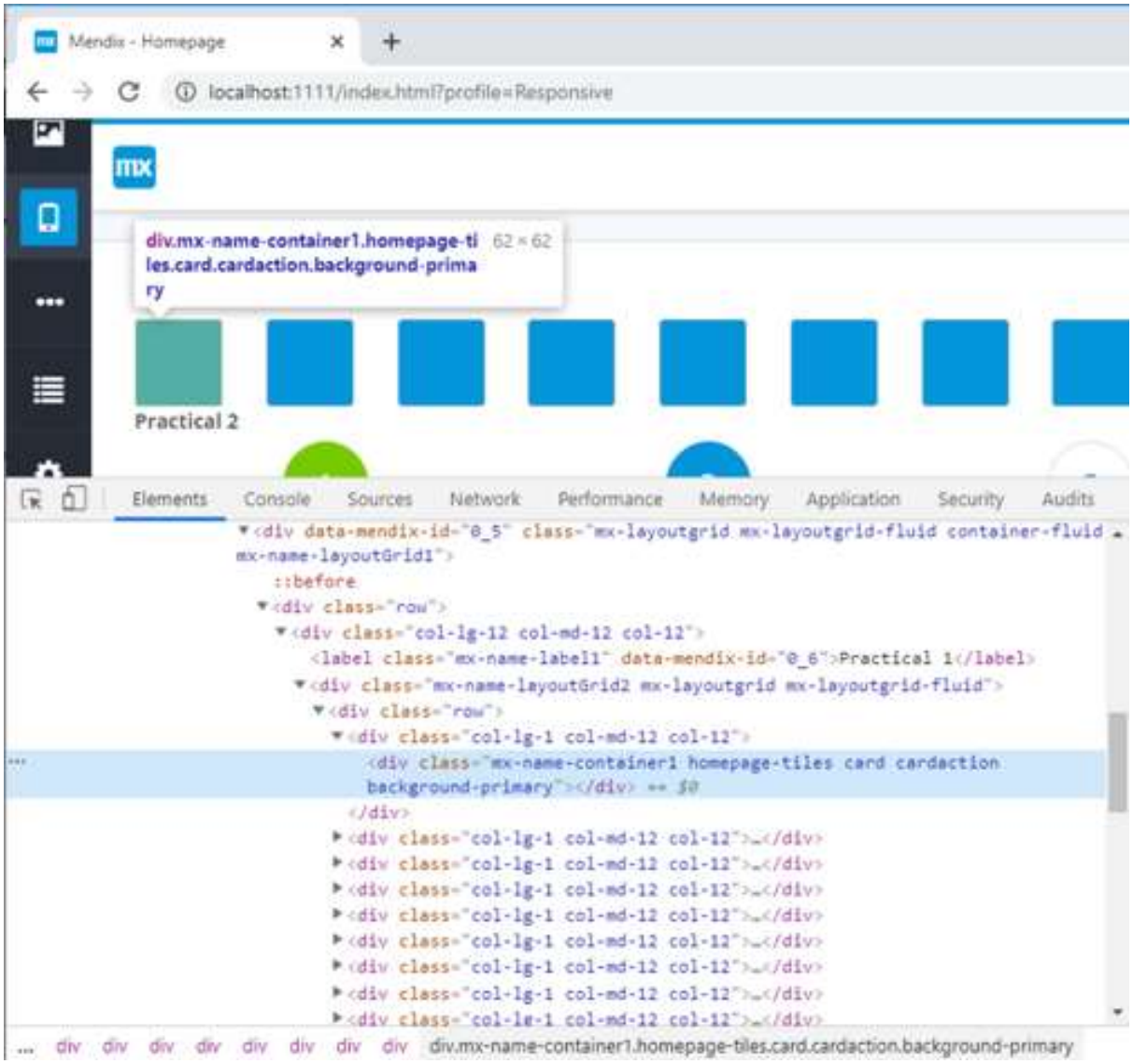


When Using the Elements Panel, your primary focus will be selecting an HTML object

We will have an opportunity to practice this in the next exercise. For now, follow along.

© Mendix Technology BV 2022. All rights reserved.

1. When you want to work with the DOM or CSS, right-click an element on the page and select **Inspect** to jump into the **Elements** panel. Or press Command+Option+C (Mac) or Control+Shift+C (Windows)
2. Hover over HTML object to see them highlighted in the browser
3. Click on arrows as an alternate way of navigating to an object
4. Double-clicking on html objects allows you to edit them

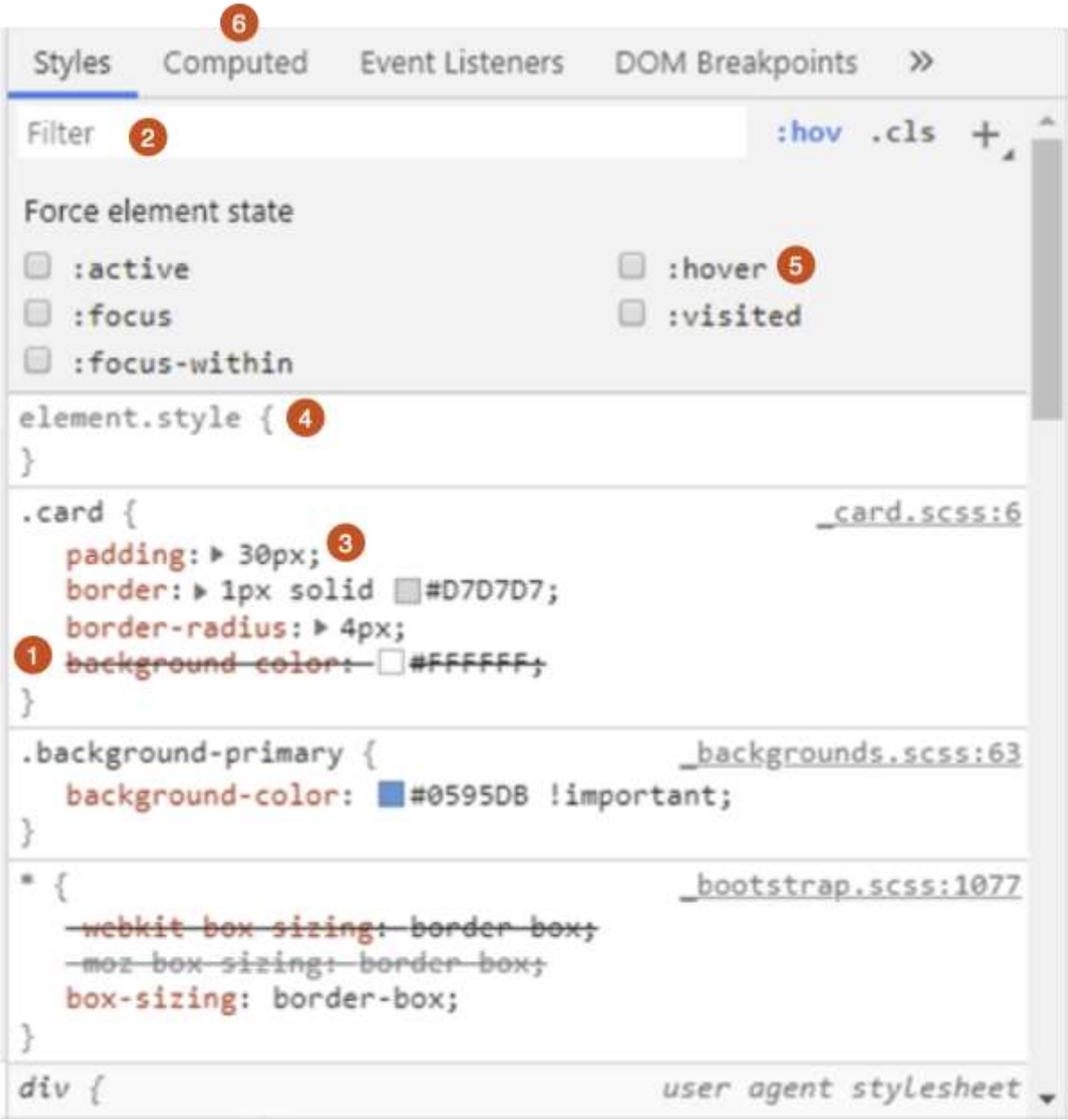


There are several features to be aware of when using the CSS Panel

With an HTML object selected you can see all the CSS classes and properties within the cascade applied to it

- Crossed through properties show that they have been overwritten
- You can search for a specific property on the filters bar
- Double-clicking on a property will allow you to change the property or value
- Element.style{} allows you to add new properties to only that object
- :Hov button allows you to toggle pseudo-states
- The Computed tab shows box model and all properties applied to it

© Mendix Technology BV 2022. All rights reserved.



Lecture 5.3.1: Change the Look of the Layout Header

In this exercise, you will take the steps we just covered and apply them to this application. For this first exercise, you will be changing the look of the layout header.

Note: Remember to have the project open and running in Studio Pro, have the SASS folder open in Visual Studio Code, and Calypso is running.

1. With the application running in Google Chrome right-click the white header and select **Inspect.**
2. The DevTools Inspector should pop up somewhere within the browser window.

Note: To change the inspector window's location, if you want to, follow the instructions in this link: *https://developers.google.com/web/tools/chrome-devtools/customize/placement*

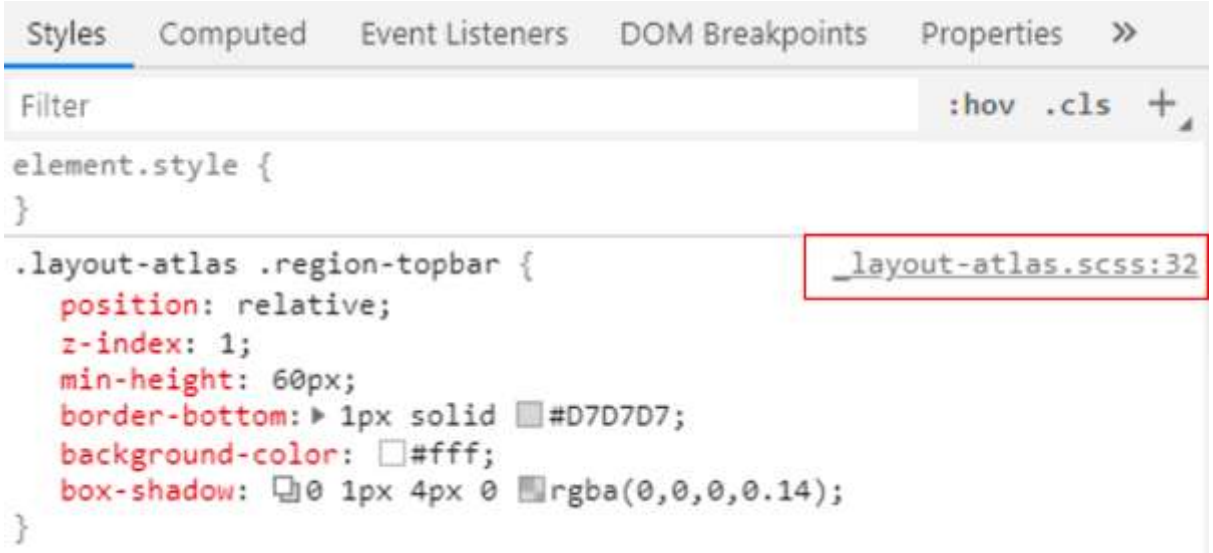
© Mendix Technology BV 2022. All rights reserved.

3. Within the Elements side of the inspector click on:

```
<div class="mx-scrollcontainer-top region-topbar">
```

you should see the CSS panel contents change.

4. Within the CSS panel at the top, you should see what properties are being applied to the selector .layout-atlas .region-topbar. to the right of the selector, you will see a partial name: number.



This partial name is the location of the partial where this Calypso code lives, and the number is the line number for this Calypso declaration. If you hover over the partial name, it will show you where in the project folder structure that this partial lives.

6. To make sure this is the right object, lets modify the background color.
7. **Click** the square between background-color and #fff. A color picker should appear, select any color to see the change. *Note: If you have trouble getting the color picker to appear you can also **click** the #fff and paste #360730.*
8. Open Visual Studio Code and navigate to the above CSS selector.
9. Now that we know the property we need to change, we can use the variables that we changed in Module 4 and apply those colors here. Change the **background-color** value to **$brand-inverse.**
10. In addition to updating the background color, we also want to remove that border at the top that spans the width of the page. Navigate to line 38 of partial _layout-atlas.scss where you should see this:

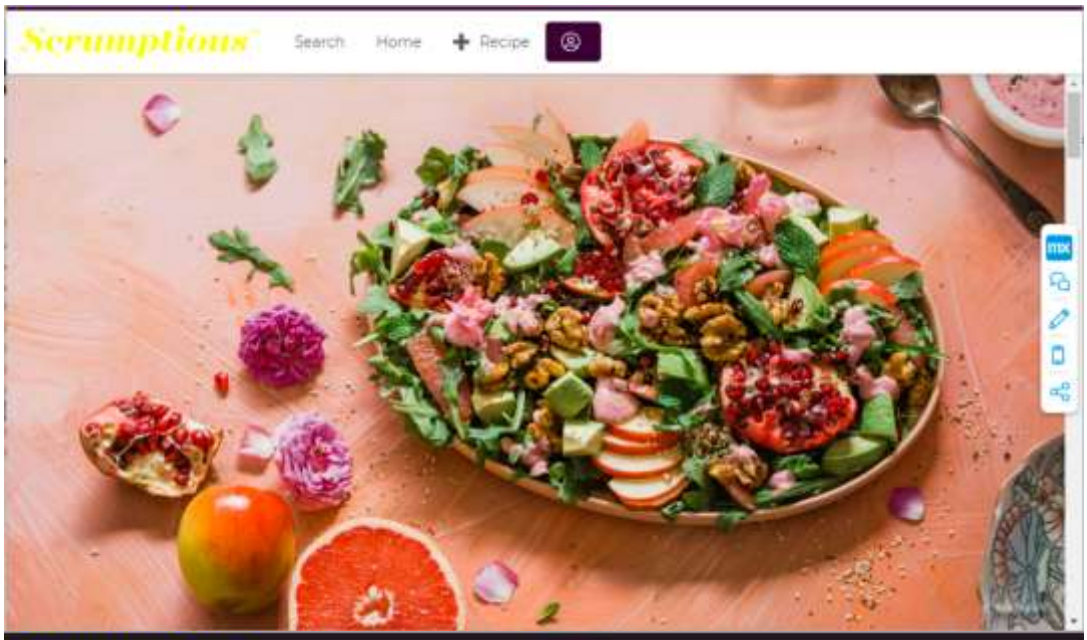
```
&::before {
```

Note: This is a pseudo-element. We use them pretty heavily in Mendix. To learn more, you can read this article: *https://css-tricks.com/almanac/selectors/a/after-and-before/*

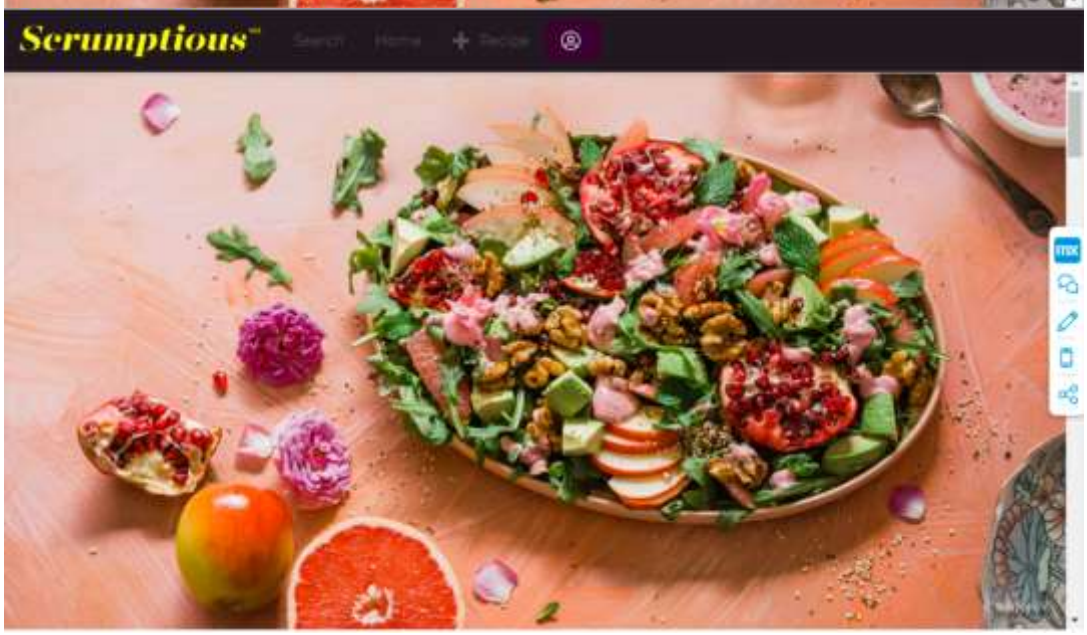
11. For our application's design, we don't have the stripe across the top. Let's delete it. Select **lines 39-36** this should include the opening and closing brackets for the :before selector and **delete.** Note: instead of deleting, you can choose to comment

© Mendix Technology BV 2022. All rights reserved.

out the styles by using // to comment out one line or / and / to comment out multiple.



Before



After

Lecture 5.4: Simple CSS rules

There are a lot of CSS declarations that are made up of 1 property and 1 value. There are a lot of topics that Calypso covers that work this way. For example, you can modify the way your text looks using these different properties:

- font-family
- Specifies the font family for text
- font-size
- Specifies the font size of text

© Mendix Technology BV 2022. All rights reserved.

- font-style
- Specifies the font style for text
- font-weight
- Specifies the weight of a font

Sometimes there you can use a shorthand, which is a CSS property that lets you set the values of multiple other CSS properties simultaneously. Instead of writing out each property listed above, you can write them in 1 line.

font: font-style font-weight font-size font-family;

In the following exercises, you are going to be modifying the styling for the headers, buttons, and navigation bar.

Lecture 5.4.1: Changing Header Styles
In this exercise, you are going to take what you learned about applying styles to text.

1. In Visual Studio Code open the typography partial in **SASS > app > widgets > _typography.scss**
2. Add the properties from the graphic spec below to the corresponding classes and save the partial

© Mendix Technology BV 2022. All rights reserved.

_typography.scss ×

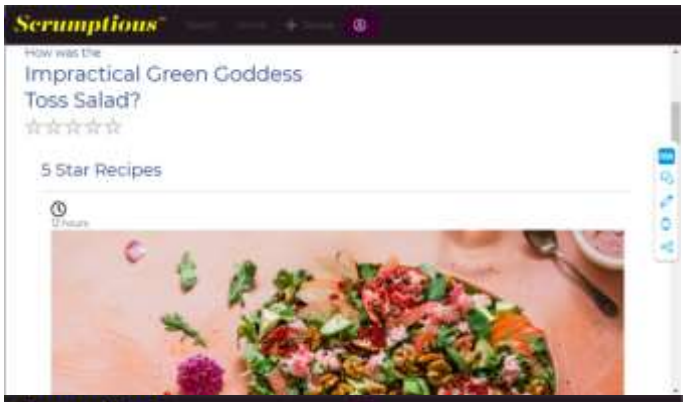
app > widgets > _typography.scss > p

```scss
/* ==================================================================
   Typography
   ================================================================== */

h1,
h2,
h3,
h4,
h5,
h6,
.h1,
.h2,
.h3,
.h4,
.h5,
.h6 {
    margin: 0;
    color: $font-color-header;
    font-weight: $font-weight-header;
    line-height: 1.3;
}

p {
    line-height: $line-height-base * 1.25;
    color: $brand-default;
}

label {
    padding-top: 0;
}

.mx-title {
    margin: 0;
    color: $font-color-header;
    font-size: $font-size-h1;
    font-weight: $font-weight-header;
}

h1,
.h1,
.h1 > * {
    font-size: $font-size-h1;
    font-family: $font-family-title;
    font-weight: 700;
    color: $brand-primary;
}

h2,
.h2,
.h2 > * {
    font-size: $font-size-h2;
    font-weight: 700;
    color: $brand-inverse;
}

h3,
.h3,
.h3 > * {
    font-size: $font-size-h3;
    font-weight: 700;
    font-family: $font-family-title;
    color: $brand-primary;
}

h4,
.h4,
.h4 > * {
    font-size: $font-size-h4;
    font-weight: 700;
    color: $brand-inverse;
}
```

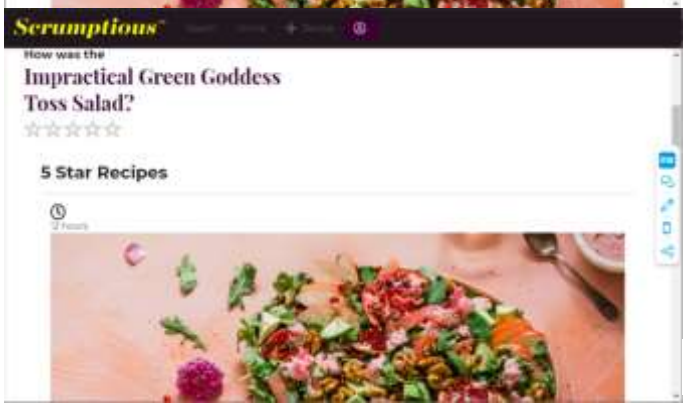
© Mendix Technology BV 2022. All rights reserved.

H1 - Impractical Green Goddess Toss Salad	font-family	$font-family-title
	font-size	$font-size-h1
	font-weight	700
	color	$brand-primary
H2 - 5-Star Recipes	font-size	$font-size-h2
	font-weight	700
	color	$brand-inverse
H3 - KFC Look-a-Like Sandwich	font-family	$font-family-title
	font-size	$font-size-h3
	font-weight	700
	color	$brand-primary
H4 - How was the	font-size	$font-size-h4
	font-weight	700
	color	$brand-inverse
P - Food qualities braise chicken cuts bowl through slices butternut snack.	color	$brand-default

Notice that that on some headers we are applying a font-family and other time not. This is because we are leveraging the cascade, for example on the H2 we are not defining the font-family, this is because the font-family: $font-family-base; is being applied at on the selector HTML tag body{}. So that by default all text elements will be set to $font-family-base unless specified otherwise. We will be seeing more examples of this in the next few exercises.



Before



After

© Mendix Technology BV 2022. All rights reserved.

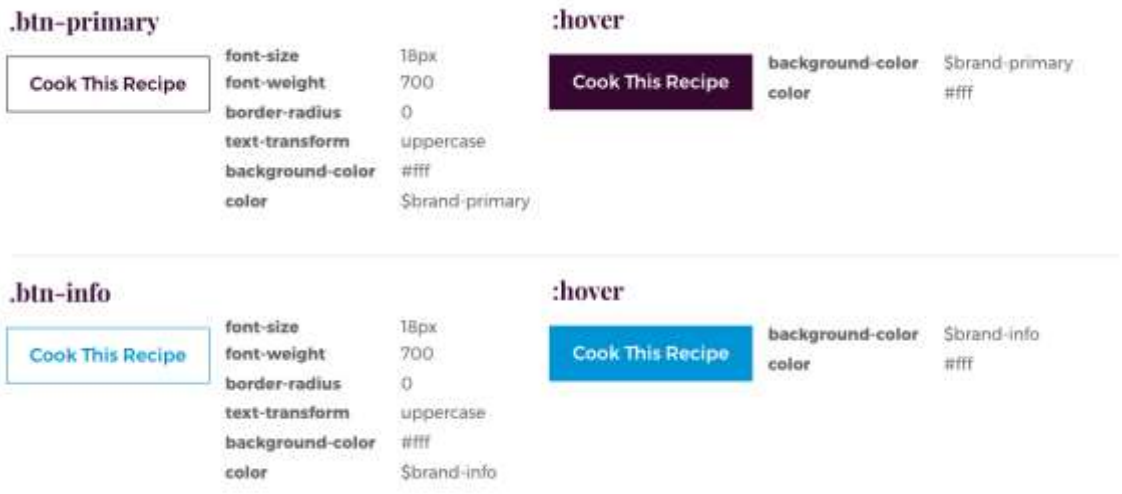
Lecture 5.4.2 Changing Button Styles

In this exercise, you will take the same principles from the typography partial and apply it to the buttons.

1. In Visual Studio Code open the typography partial in **SASS > app > widgets > _buttons.scss**
2. In the buttons partial, observe how the styles are presented on the page. The broader, more general classes are defined at the top.

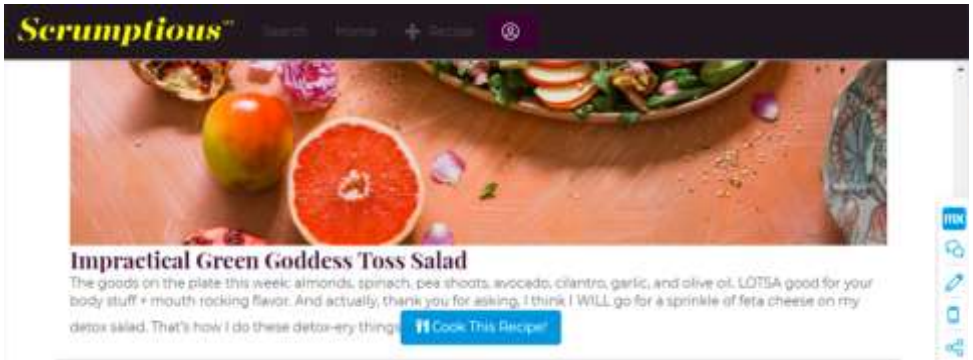
.btn, .mx-button {}
Those two classes are applied to every button in all Mendix applications, and then more specific classes are declared at the bottom. This pattern of broad to specific classes can be found across the partials in Atlas UI and are a good showcase of how to leverage the cascade built into CSS.

3. First, we will update the broader styles of .btn, mx-button{}
4. Take a look at the graphical spec below and take note of which styles are shared across buttons. The graphic shows how the buttons look on the page and :hover shows how they will look when you move your mouse over the button.

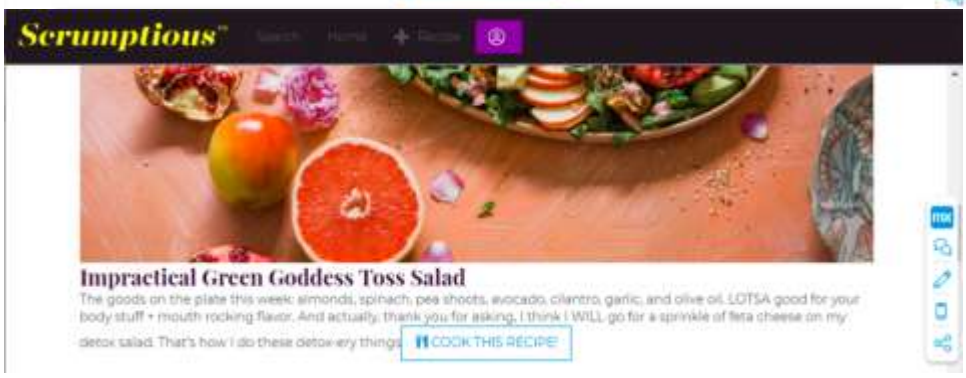


5. Apply the shared styles from the graphic in the **.btn, mx-button{} selector**
6. On lines 60 - 66, you should see two @include button-variant(). These are whats known in SASS as mixins, but in other development languages closely resemble functions. These mixins allow you to pass in CSS properties, and the mixin will export code. In this case, it will create different looking buttons based on the colors you pass in. Use the graphical spec to define the arguments for **btn-primary** and **btn-info**

© Mendix Technology BV 2022. All rights reserved.



Before



After

Lecture 5.4.3: Changing Navigation Styles

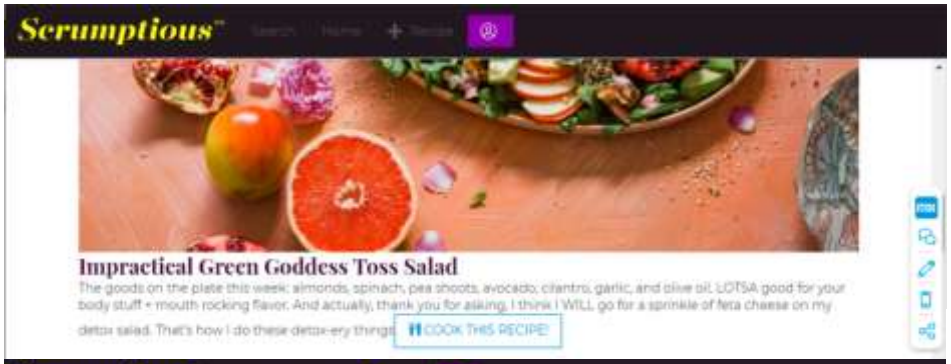
In this exercise, you will make the text in the navigation stand out as it does in the design below.

1. With the application running in Google Chrome right-click a navigation element and select **Inspect.**
2. In the CSS Inspector panel, change the text **color to #fff.**
3. Click to the right of the font-size property to add a new property to the styling selector. Add **text-transform** to make the text uppercase and **font-weight** to make the text bold. Copy the new and changed style properties in the inspector.
4. Navigate to the partial and line number in VSCode, as shown on the Inspector panel.
5. Paste the copied Calypso properties within the styling selector.

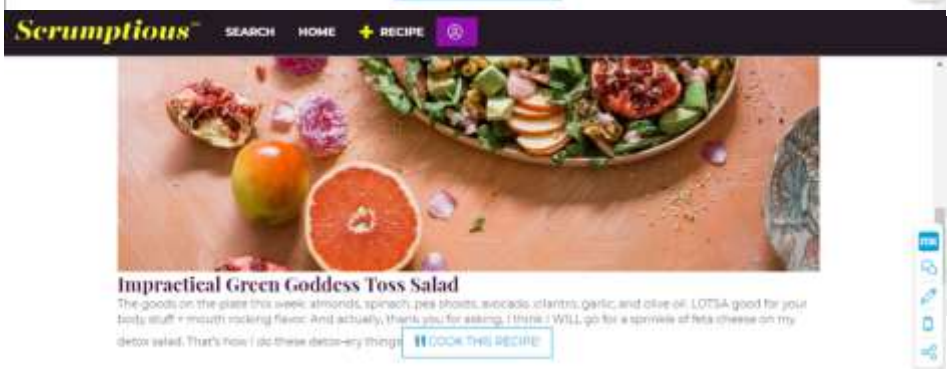
Note: if you want to update the background color and text color of the navigation on hover, you could do that in this partial. For now, we will be leaving it as it is.

6. The last change we are going to make is to change the plus icon to yellow.
7. With the application running in Google Chrome, right-click the plus icon and select **Inspect.**
8. In the CSS Inspector panel, click to the right of the font-size property to add a new property to the styling declaration. Add a **color** property to make the icon yellow.
9. Copy the new property and navigate to the partial and line number in VSCode as shown on the Inspector panel.
10. Paste the copied CSS properties within the styling selector.

© Mendix Technology BV 2022. All rights reserved.



Before



After

Lecture 5.5: Summary

Congrats! In this module, you learned about the steps and tools needed to do CSS development with Mendix. You also applied some simple Calypso rules to your application.

Module 6: The Box Model

In the last module, we covered how to apply CSS within Mendix and applied a few simple CSS rules. In this module we will cover a fundamental concept for the composition of HTML web apps.

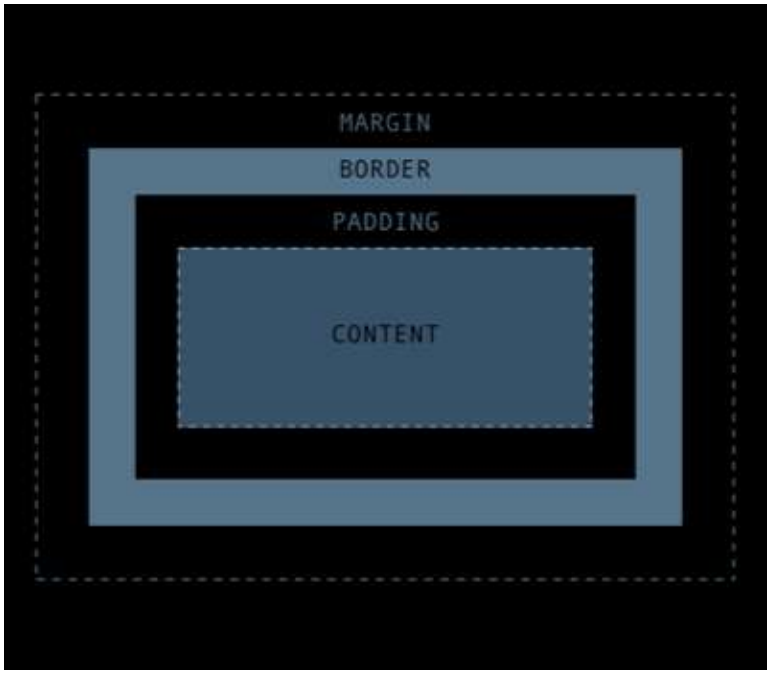
Lecture 6.1: What is the Box model

If you were to turn off the CSS on any web app, the result would be content stacked one on top of the other, so in essence, every object on the page is a box. Knowing how to leverage the properties of these boxes is key to creating layouts with CSS, or aligning items with other items.

The box model is made up of 4 properties:

1. The content dimensions, width, and height, of an element's box.
2. The paddings, or inner spacing, of an element's box.
3. The borders, outside perimeter, of an element's box.
4. The margins, or outer spacing, of an element's box.

© Mendix Technology BV 2022. All rights reserved.



Lecture 6.2: Box Behaviour

The property of **Display** has an impact on how the box behaves in terms of page flow, and its relation to other boxes on the page. The three values we will be covering here are Block, Inline, and Inline-Block.

Block

- Block-level elements *block* the full width of their parent elements
- They prevent other elements from appearing in the same horizontal space
- The width & height property can also be set

Inline

- Inline elements only take up the amount of space necessary to display their content
- The height and width of these elements cannot be changed in the CSS
- Inline elements will sit side by side

Inline-block

- Inline-block display combines features of both inline and block elements.
- Inline-block elements can appear next to each other
- their width and height properties can be set

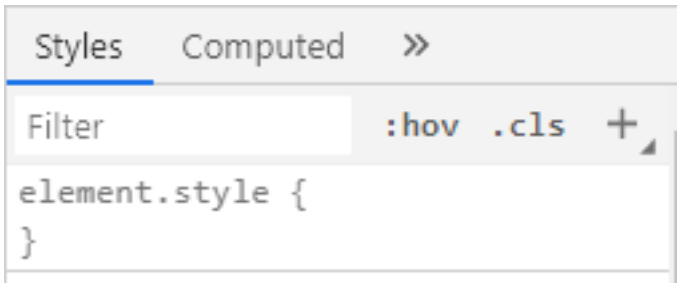
© Mendix Technology BV 2022. All rights reserved.

Lecture 6.2.1: Modifying Box Behavior

In this exercise, you will change the display value to make two items sit next to each other.

Note: make sure you have completed steps 1-5 of Exercise 4.1 so that these new changes will be incorporated into the compiled CSS.

1. In Studio Pro, **open** page **Recipe_Home** and navigate to the listview below the header that says 5 Star Recipes.
2. **Select** the container that is wrapping the clock icon and minutes text. Add a class to the class field called "**time-tag**".
3. Click **Run Locally** to apply the application changes to Google Chrome.
4. R**ight-click** the clock icon and select **Inspect** to open your browser inspector window**.**
5. On the styling side of the browser inspector, there should be a selector that says "element.style{}". This is where you can add or see any inline styling for the selected element.



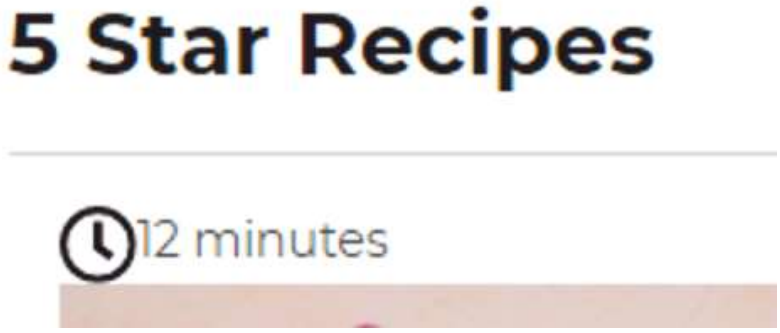
6. Click to the right of the open bracket to add a new property to the styling declaration. Add **display: inline-block** to make the minutes text sit next to the clock image. Copy the new style properties in the inspector.
7. Open VSCode and open the **_scrumptious.scss** partial add the following nested class selectors:

```
.time-tag{
img.mx-image{

}
}
```

8. Within **img.mx-image{}** paste the copied CSS property. Save the file so that the changes can be applied to the browser.

© Mendix Technology BV 2022. All rights reserved.

Before

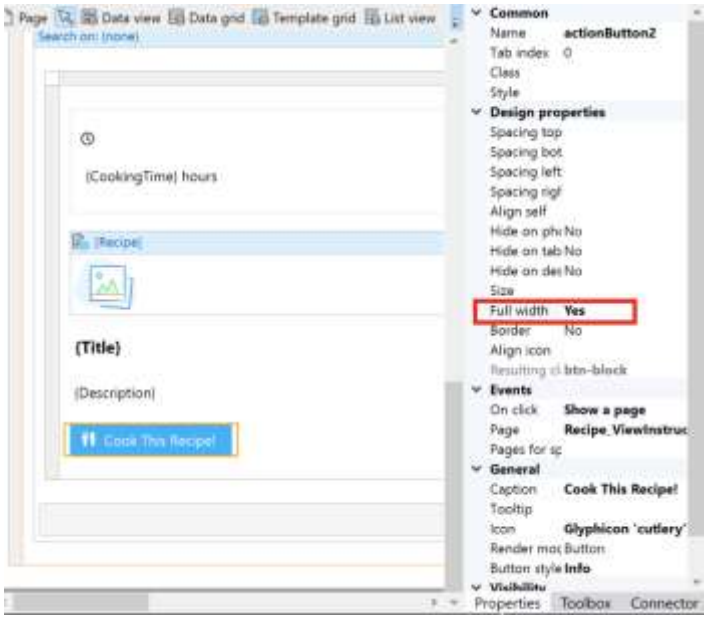


After

Lecture 6.2.2: Box behavior design properties

Changing the display property is a powerful and commonly used practice. Due to its usefulness mendix has included it in several of its design properties. A design property is an alternative way to add a class to a mendix object. In this exercise, you will put a design property on a button.

1. In Studio Pro, open page **Recipe_Home** and navigate to the listview below the header that says 5 Star Recipes.
2. Select the button that says “Cook this Recipe”.
3. In the **Properties** pane, change the **Full width** property to **Yes**.



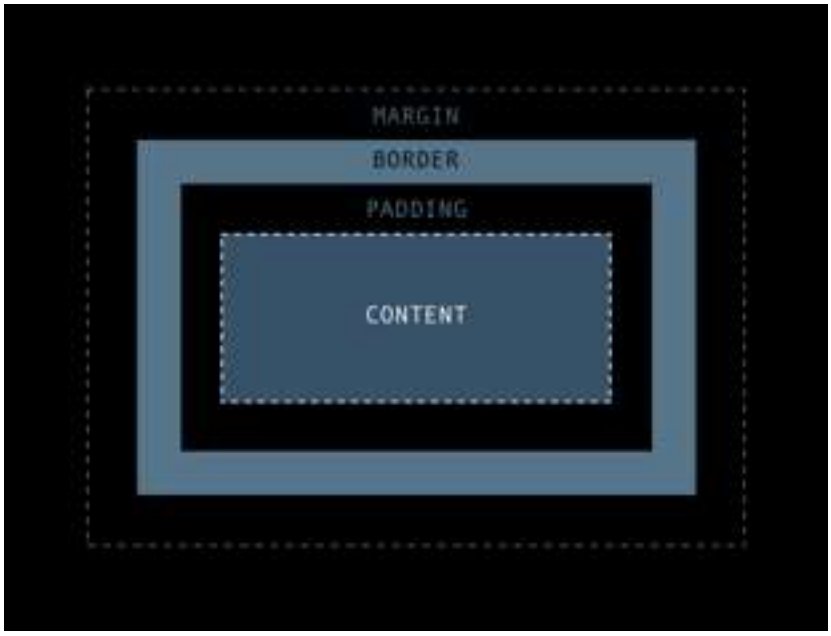
© Mendix Technology BV 2022. All rights reserved.

5. Click **Run Locally** to see changes in the browser.

Lecture 6.3: Height & Width

If the element display is set to anything other than inline, then the properties height and width can be set. Within CSS, there are numerous types of measurement units available. Below is a list of commonly used ones:

- **px**: pixels allow you to set an item to an exact size
- **%**: Defines a measurement as a percentage relative to another value, typically an enclosing element.
- **em**: A relative measurement for the height of a font in em spaces.



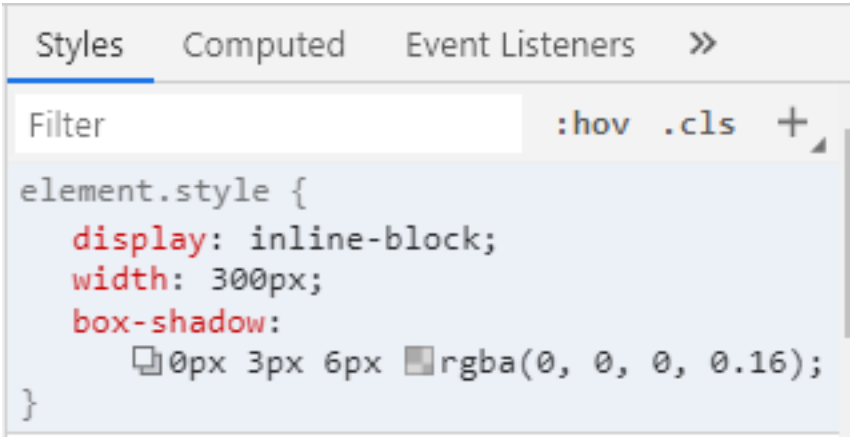
Lecture 6.3.1: Modify Height & Width

In this exercise, you will be updating the properties for the list of recipes so that they will appear next to each other.

1. In Studio Pro, open page **Recipe_Home** and navigate to the listview below the header that says 5 Star Recipes.
2. Select the **listview** and add a class to the class field called **horizontal-list**.
3. Click **Run Locally** to apply the application changes to Google Chrome.
4. Right-click the **listview** and select **Inspect** to open your browser inspector window.
5. In the **Elements** tab of the inspector panel, select the tag with the class **horizontal-list** applied.
6. To change how the items in the list display, you need to target the individual list items, within HTML this is represented by a **li tag**. Select a **li tag** within the horizontal-list.

© Mendix Technology BV 2022. All rights reserved.

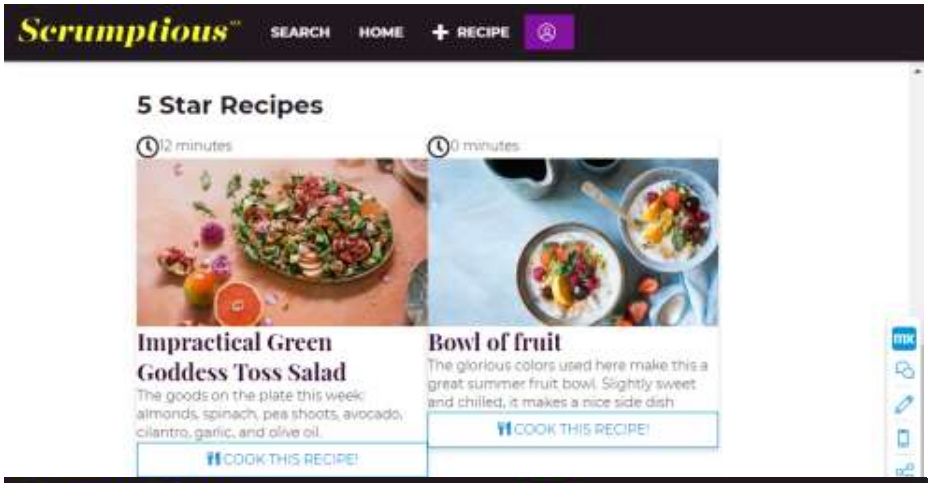
7. In the "**element.syle{}**" under the styles panel, add the following properties:



Note: by using the **element.syle{}** selector, you will only see the changes applied to 1 element.

8. Copy the properties from the Inspector panel.
9. Open VSCode and navigate to **app > widgets > _listview.scss**. Within this partial, you'll see a selector for **.listview-stylingless**. Change the selector listview-stylingless to horizontal-list so that the selector looks like **.mx-listview.horizontal-list**. In this case we are repurposing an existing class as it is an easier baseline to start from.
10. At the bottom on the **& > ul > li {** selector paste the copied Calypso properties. **Save** the file and switch to the browser window to see the applied changes.

© Mendix Technology BV 2022. All rights reserved.



Before

After

Lecture 6.4: Padding

Padding is defined as the space between the contents of a box and the borders of a box. It is most often used to expand the background-color and make the content look less cramped. When applying the property of padding, border, and margin you have the option to apply values to a specific side property:

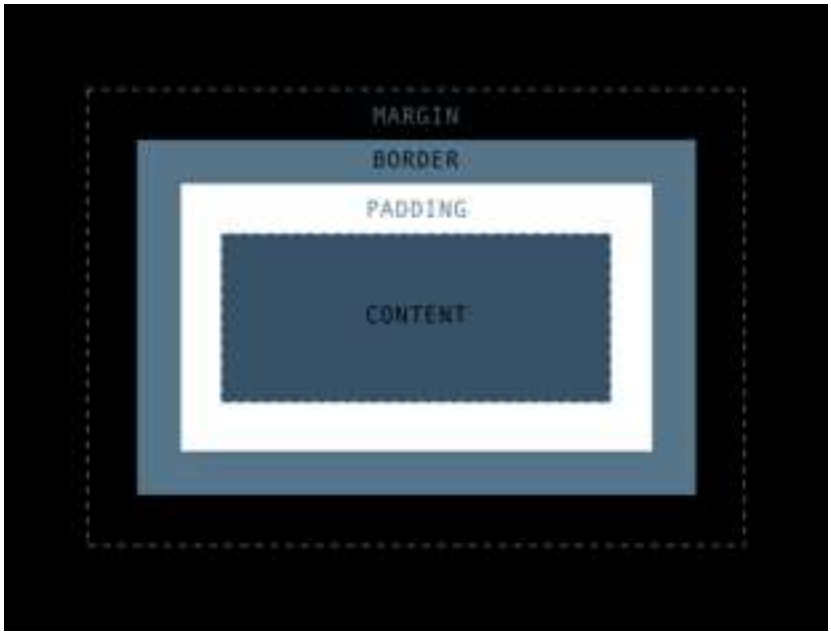
- **padding-top**
- **padding-right**
- **padding-bottom**
- **padding-left**

You can also use the property **padding** that combines the above properties into one property. This shorthand accepts one, two, three, or four, values.

- When **one** value is specified, it applies the same padding to **all four sides**.

© Mendix Technology BV 2022. All rights reserved.

- When **two** values are specified, the first padding applies to the **top and bottom**, the second to the **left and right**.
- When **three** values are specified, the first padding applies to the **top**, the second to the **right and left**, the third to the **bottom**.
- When **four** values are specified, the paddings apply to the **top**, **right**, **bottom**, and **left** in that order (clockwise).

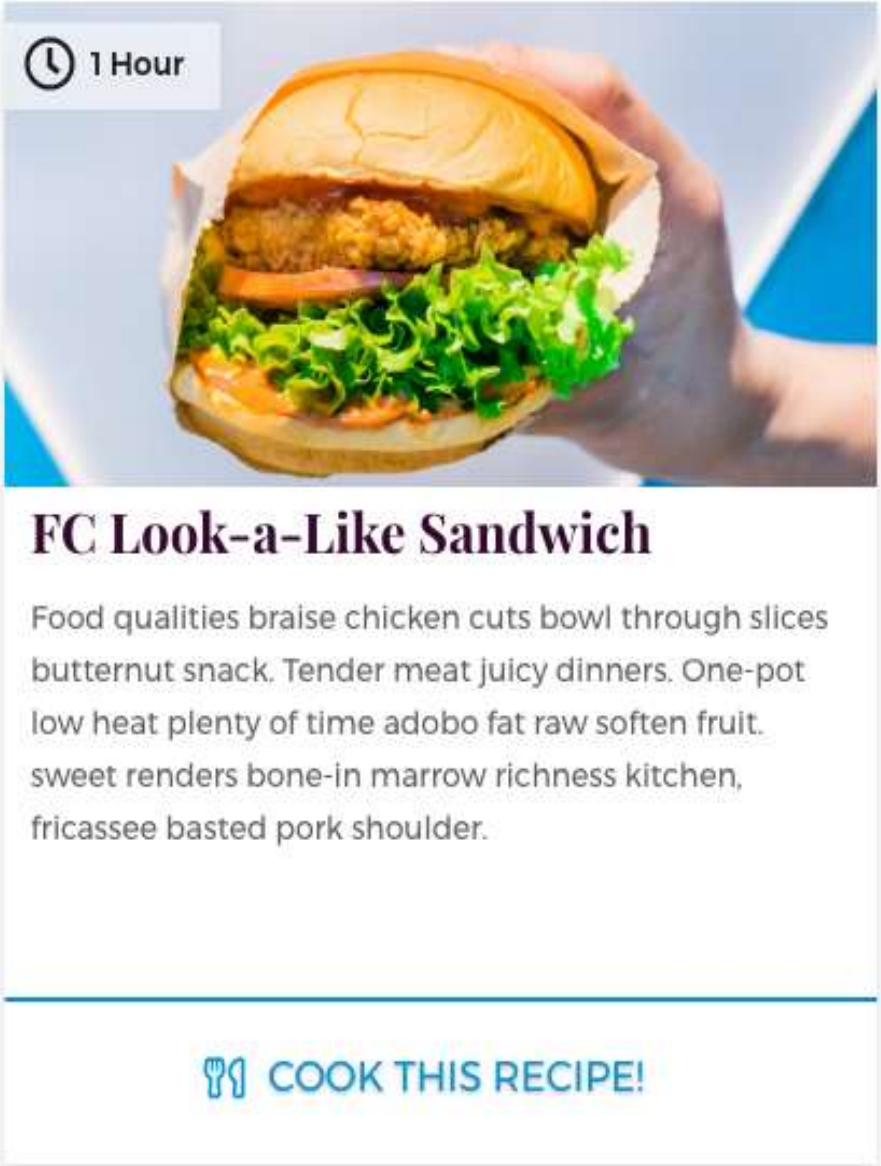


Lecture 6.4.1: Applying Padding

In this exercise, you will use what you learned about padding to make the content within the recipe listview not be so squished. Within the listview widget below the image, there are two text objects, a title, and description. Based on the page design mockup you are going to add a white space between the text and edges of the list item.

1. In Studio Pro, open page **Recipe_Home** and navigate to the listview below the header that says 5 Star Recipes.

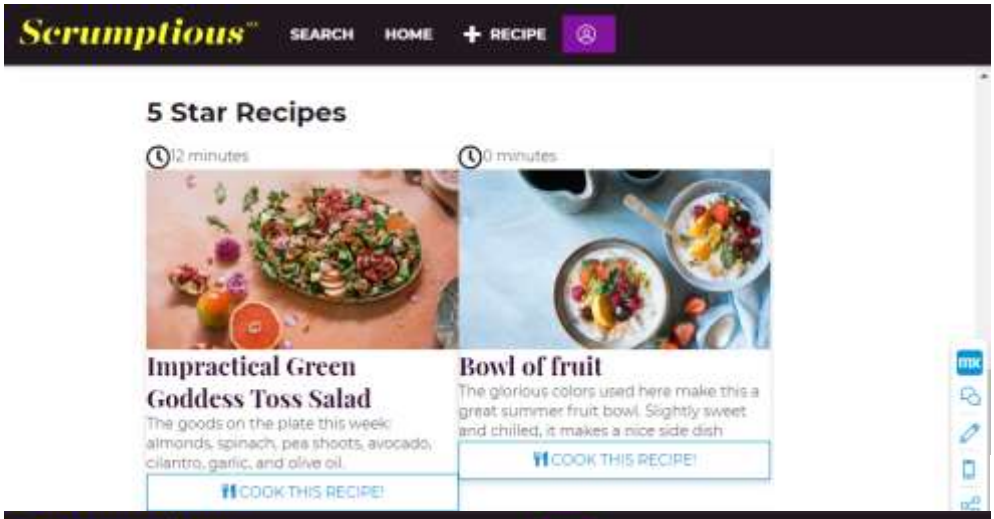
© Mendix Technology BV 2022. All rights reserved.



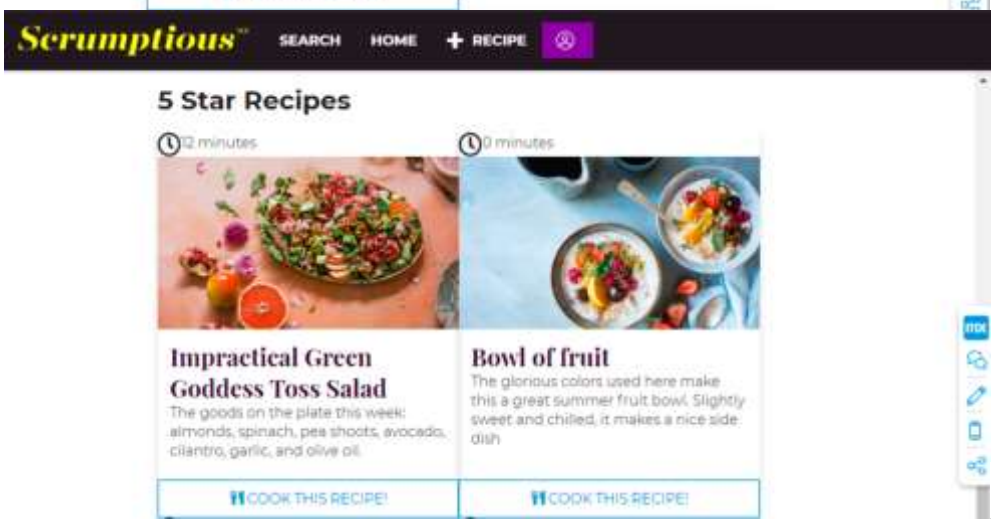
2. Right-click the title text widget and select **Wrap in container**. Drag the **description text** element and place it in the newly created container.
3. With the container selected, go to the **class** field of the **Properties** pane and add input card-content to the field.
4. Click **Run Locally** to apply the application changes to Google Chrome.
5. Right-click the **recipe title** and select **Inspect** to open your browser inspector window.
6. Navigate to and select the tag with class card-content applied.
7. n the CSS panel, use the padding shorthand to give the class **12px of padding on all sides**. Copy the styles.

© Mendix Technology BV 2022. All rights reserved.

8. Open VSCode and navigate to the **_listview.scss** partial under the apps folder. You need to add the class selector of **.card-content** within the **horizontal-list{ &>ul>li {** selectors. Note: When working with nested selectors be careful where you are adding your selector or styles.
9. Within the **.card-content** class paste the padding styles. Also add a property of **height:150px**. Save the file and switch to the browser window to see the applied changes.



Before



After

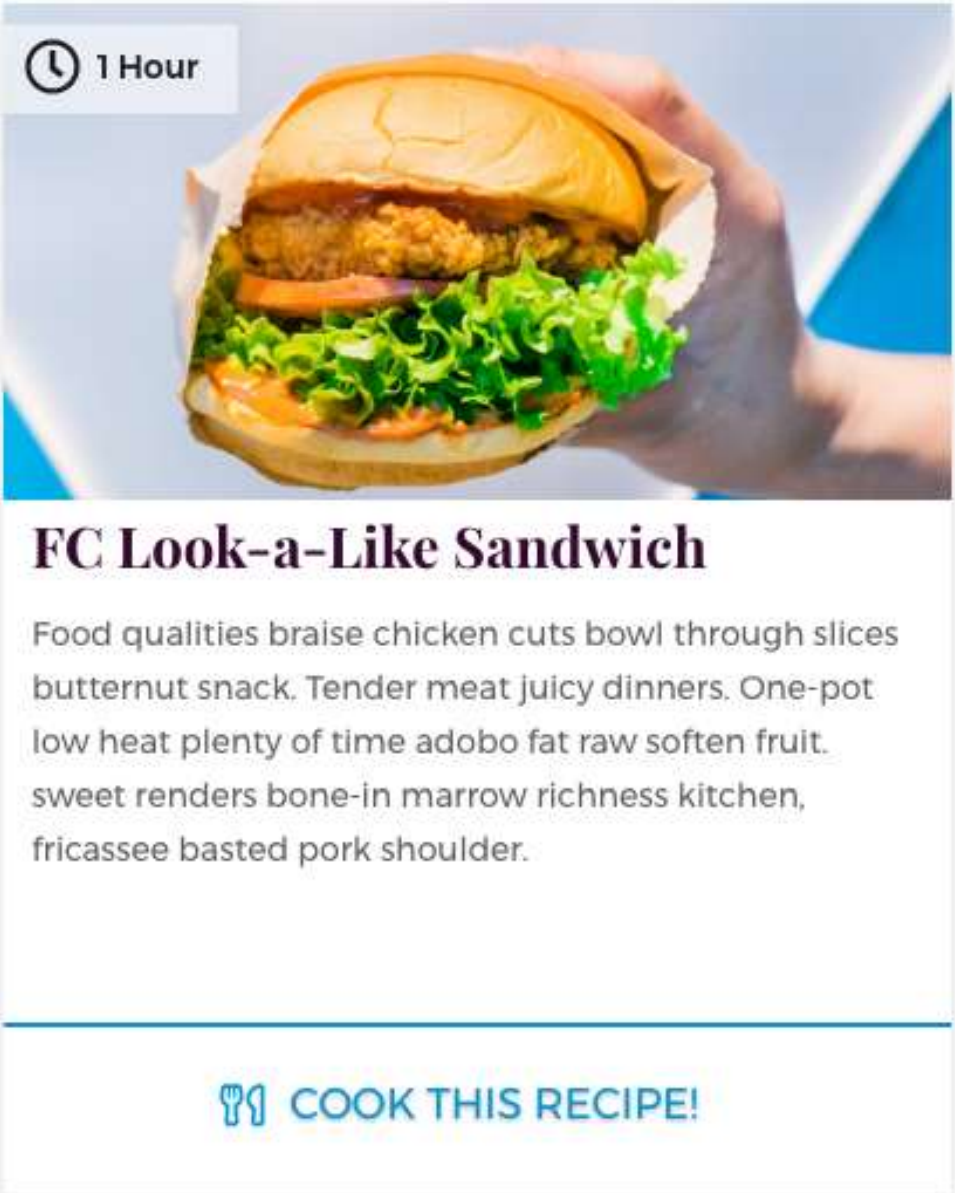
© Mendix Technology BV 2022. All rights reserved.

Lecture 6.5: Border
A border is a line that surrounds an element. Borders can be set with a specific width, style, and color.

- **width**: use pixels to define the thickness of the border
- **style**: Defines how a border will look. Browsers can render borders 10 different styles
- **color**: can use any format your comfortable with. Most common format is the hex code.

Lecture 6.4: Border

In this exercise, you are going to use what you learned about borders to update the styling of the cook button to match the design.



1. In Studio Pro **open** page **Recipe_Home** and navigate to the listview below the header that says 5 Star Recipes.

© Mendix Technology BV 2022. All rights reserved.

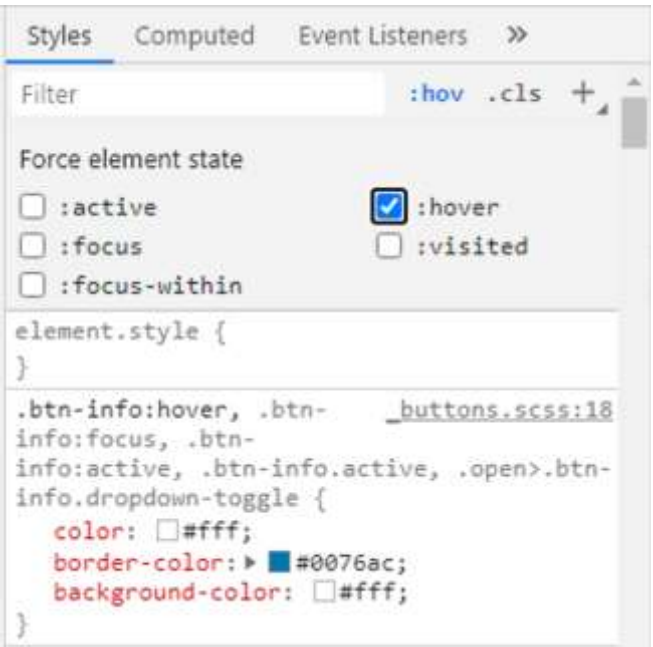
2. Within the list view widget, navigate to the “Cook this recipe!” button and look at its properties. Note that the Button style is set to Info.
3. Open VSCode and navigate to the **_buttons.scss partial** under the **apps > widgets** folder.
4. Find the line that has a similar name to the button style.

```
.btn-info {
    @include button-variant($brand-info, #fff, $btn-info-border-color, $btn-info-bg-hover);
}
```

Note: This is an example of a Sass mixin as mentioned in lecture 4.2. For our use case we don’t want the info button to be styled like all the other buttons. But instead of writing out all the styles we are going to just overwrite the properties we want to change.

5. Within the **.btn-info selector** and below the @include add the following properties **border:none;** and **border-top:1px solid $brand-info;** Save the file and switch to the browser window to see the applied changes.
6. You might notice that there is a small problem with visibility when you hover/ move your mouse over the button. The button text looks like it disappears. To see what styles are applied when you hover over the button, **Right-click** on the button and select **Inspect** to open your browser inspector window.
7. On the Styles panel **click** the text **:hov,** to toggle the elements states.

8. With the :hover state checked you can see what styles are applied to the hover state. The color property, the text color, is set to white so that’s why is seems disappear when you mouse over.



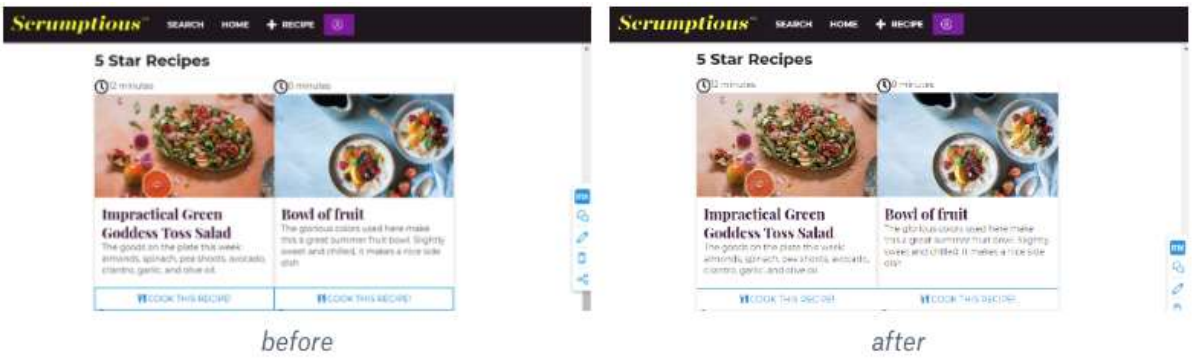
© Mendix Technology BV 2022. All rights reserved.

9. Let's swap the **background-color** and **color** property values. In the styles panel change the **color** property from #fff to #0076ac and change the **background-color** from #0076ac to #fff. Copy the css styles.
10. Return to the **.btn-info** selector in the **_buttons.scss** partial. Add the following declaration under the **border-top property**

```
&:hover,
  &:focus,
  &:active,
  &:active:focus {

  }
```

Note: This selector will allows you to account for the other states that were seen in the inspector, which will make the styling for the button look good at all interaction points.

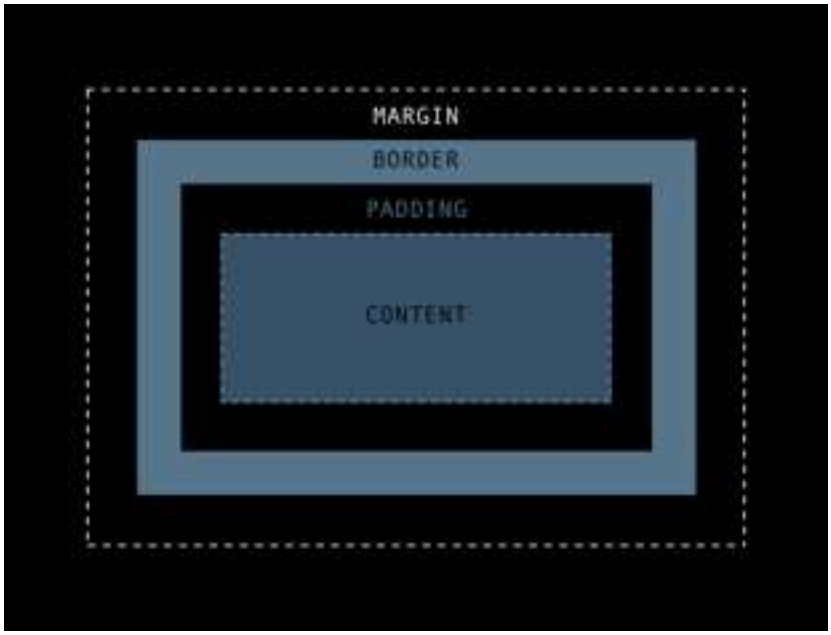
11. Paste the styles from the inspector and save.



Lecture 6.6: Margin

Margin refers to the space directly outside of the box. Margin shares the same set of properties and shorthand as border and padding. Margin's are primarily used to make space between objects. You can also use the margin property to center content.
By using
margin: 0 auto;
you can horizontally center an object within it's parent object. However, the above declaration will only work if the parent element has a set width.

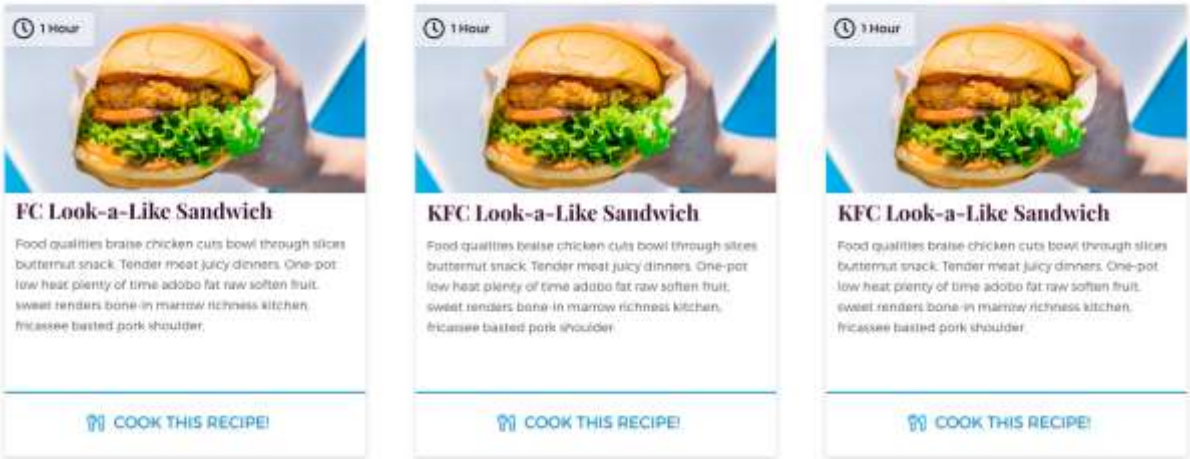
© Mendix Technology BV 2022. All rights reserved.



Lecture 6.6.1: Using the Margins Property

In this exercise, you will be using the margin property to give the recipe list space between recipes.

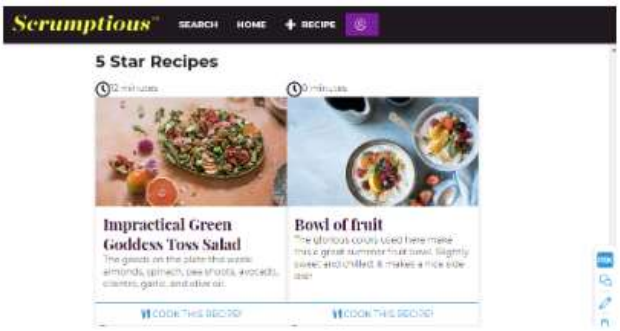
5-Star Recipes



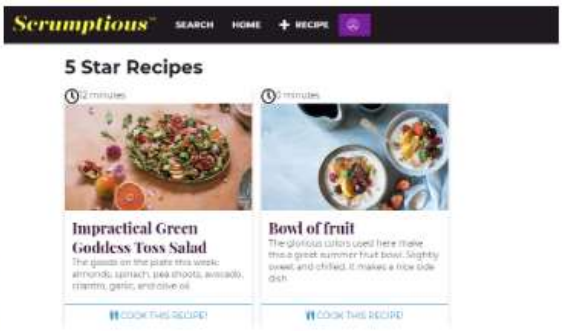
1. With the application running in Google Chrome, **right-click** a recipe list item and select **Inspect.**
2. In the **Elements** Inspector panel, navigate to and select a **li tag.** You should see the styles panel change to display .**horizontal-list.mx-listview>ul>li { selector** and corresponding styles.

© Mendix Technology BV 2022. All rights reserved.

3. Click to the right of the box-shadow value to create a new CSS property line.
4. Add **margin-right: 15px** CSS property/value. Copy the new CSS property.
5. Use the partial name and number displayed to the right of the CSS selector, to find where in the VSCode you need to navigate to paste the copied property.
6. Paste the CSS property into the partial and save.



before



after

Lecture 6.6.2: Margins Design Properties

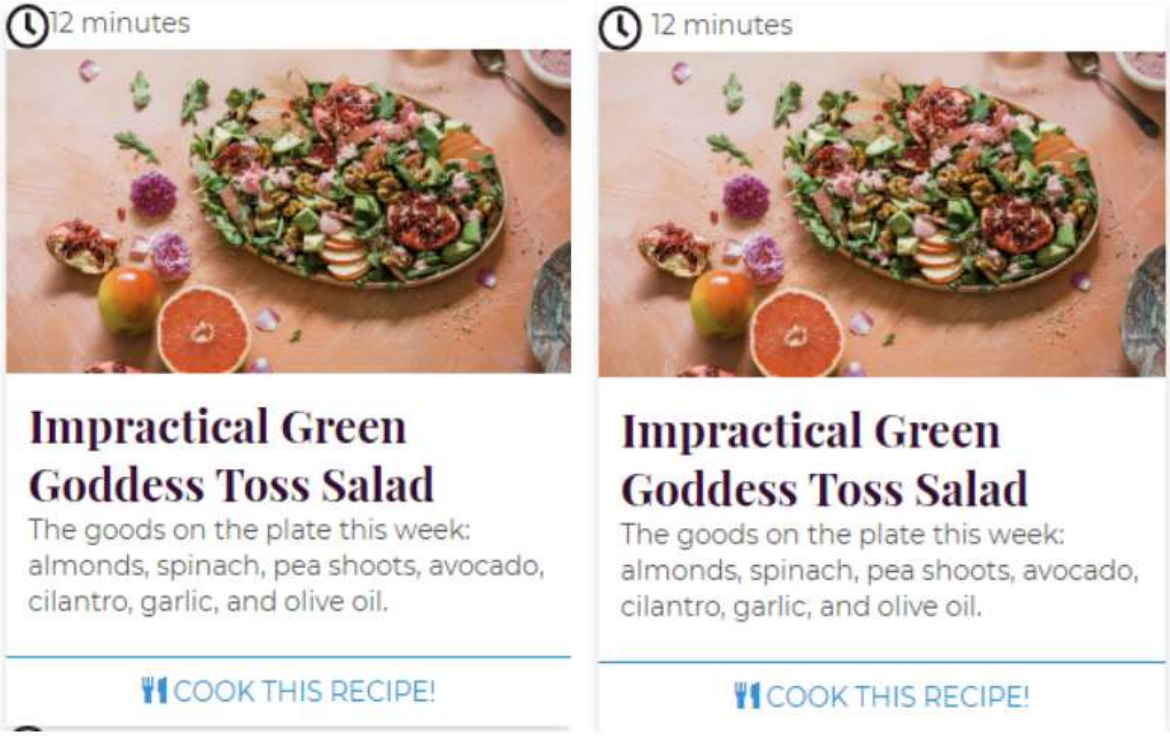
In this exercise, you will use the spacing-right Design Property to give the clock image and time text some space.

1. In Studio Pro, open page **Recipe_Home** and navigate to the container with the applied class time-tag.
2. Select the clock image. In the **Properties** pane, under the Design properties heading, navigate to the **spacing-right** dropdown and select **Small**. If you look at the label that says resulting class you should see **spacing-outer-right** now. When you use Design properties you are also applying classes to widgets on the page.

Note: All the spacing properties have the same options: None, Small, Medium, and Large these selections correspond to 0px, 5px, 15px, 30px respectively.

© Mendix Technology BV 2022. All rights reserved.

3. Click **Run Locally** to see the changes applied in the browser.



before *after*

Lecture 6.7: Overflow

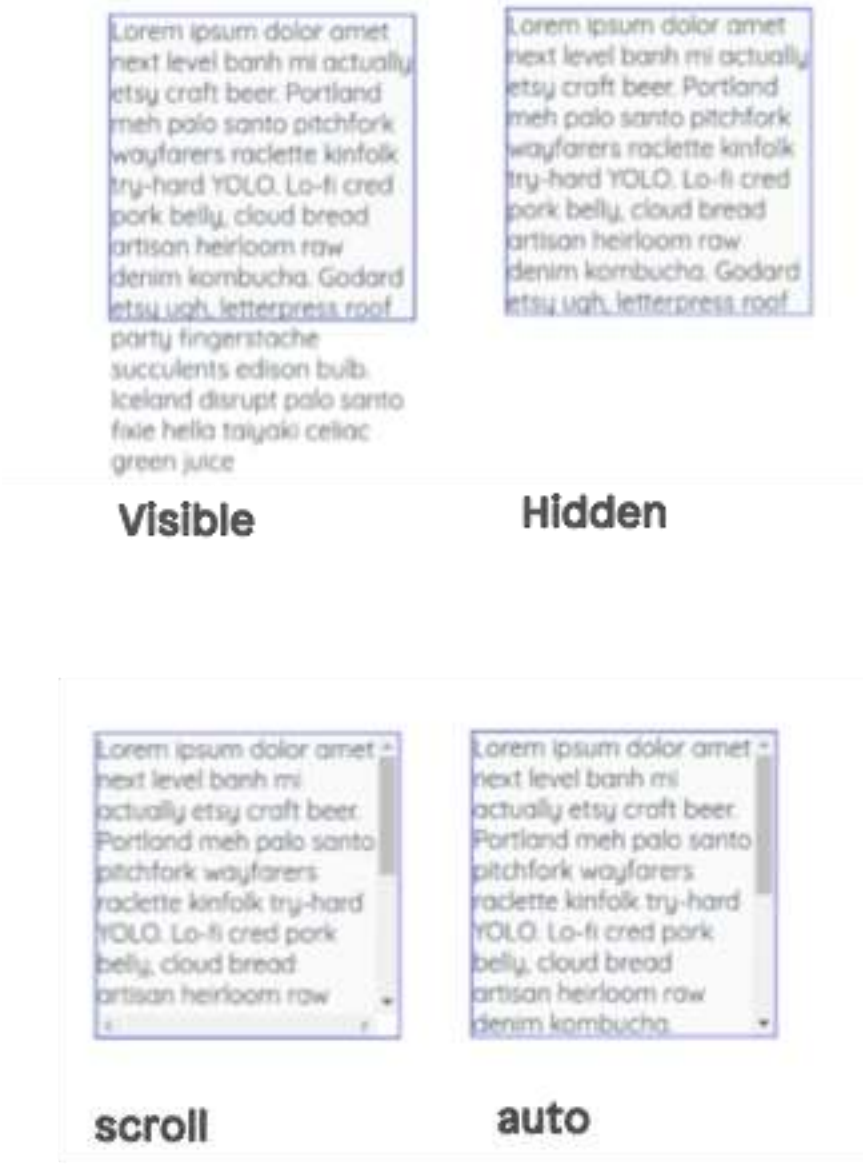
Overflow is a CSS property that controls what happens to content that spills, or overflows, outside its box. There are three variations of the overflow property:

- Overflow-x pertains only to horizontal or x-axis of the object
- Overflow-y pertains only to vertical or y-axis of the object
- Overflow affects both x and y-axis

The overflow properties have four commonly used values:

- **visible**: when set to this value, the overflow content will be displayed outside of the containing element
- **hidden**: when set to this value, any content that overflows will be hidden from view
- **scroll**: when set to this value, a scrollbar will be added to the element's box
- **auto**: when set to this value, a scrollbar will be added to the element's box on the y-axis when the content exceeds the box

© Mendix Technology BV 2022. All rights reserved.

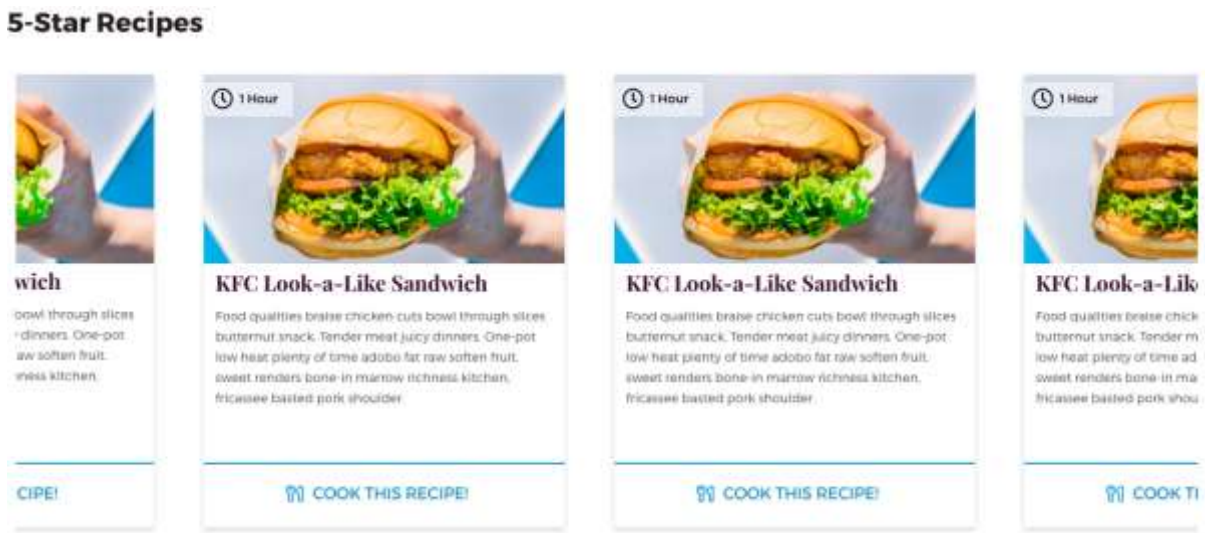


The overflow property doesn't apply only for text, but any content of a container, such as a list.

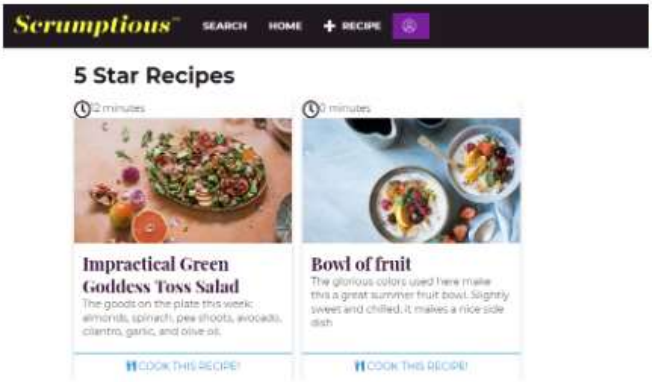
Lecture 6.7.1: Overflow

In this exercise, you will be using the overflow property to make the recipe list scroll horizontally.

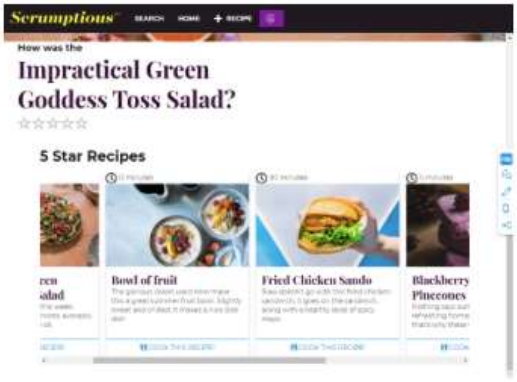
© Mendix Technology BV 2022. All rights reserved.



1. With the application running in Google Chrome, right-click a recipe list item and select **Inspect**.
2. In the **Elements** Inspector panel, navigate to and select the div with class mx-listview. You should see the styles panel change to display **.mx-listview{ selector** and corresponding styles.
3. Click to the right of the **padding** value to create a new CSS property line.
4. Add **overflow-x:auto**; CSS property. Copy the new CSS property. Now you will probably notice that nothing happened. This is because, by default, the content wants to flow down, not horizontally. To encourage the content to flow horizontally, you need to give it space to do so by giving the wrapper container a set width.
5. First, switch to VSCode and navigate to **app > widgets > _listview.scss** and paste the overflow property after the **.mx-listview{ declaration**. Switch back to the browser.
6. Determining where to add the width is the next challenge. Try adding **width:1600px** to the **.mx-listview{** selector to see what happens. You want only the recipe cards to be wrapped in a scrolling container. If the **.mx-listview{** selector doesn't work, remove the applied property, and try applying it to the **ul** tag.
7. Once you find under what selector the width should be applied, switch to VSCode. Navigate to **app > widgets > _listview.scss** and add the selector within **.mx-listview{** selector and add the styling declaration **width:1600px** to it. **Save** the partial and switch to Chrome to see the changes.



before



after

© Mendix Technology BV 2022. All rights reserved.

Lecture 6.8: Summary

In this module, you learned about the fundamental concept of the Box Model and its effect on how objects are displayed in the browser.

Module 7: Positioning

I the last module we covered the concept of the box model. In this module we will learn about how we can change the placement of objects on the screen.

Lecture 7.1: Flow of HTML

A browser will render the elements of an HTML document that has no CSS from left to right, top to bottom, in the same order as they exist in the document. This is referred to as the flow of the page.

CSS includes properties that change how a browser *positions* elements on the page. These properties allow us to move an element out from the flow of the HTML page. There are four properties we are going to cover to change the position of elements on a page:

- position
- z-index
- float
- clear

Lecture 7.2: Position

Setting an objects position property to a value of relative allows you to modify the default position.

There are three values regarding the position property we are going to cover:

- relative
- absolute
- fixed

Relative
an element's original position remains in the flow of the document, but you can use any combination of the **positioning properties**:

- top
- right
- bottom
- left.

© Mendix Technology BV 2022. All rights reserved.

These properties accept positive and negative number values with px, em, or other units of measure values. They allow us to move the element from its original position in the direction of the selected position property.

Absolute
When an object's position is set to absolute, the object is removed from the flow of the document, and other elements will behave as if it's not even there. For example, if you see an object stacked on top of another object, it is likely that its position is set to absolute. Then you can use the positioning properties on this object to set its location. When used in tandem with a parent container set to relative, the position of the object will be related to where the parent object exists on the page.

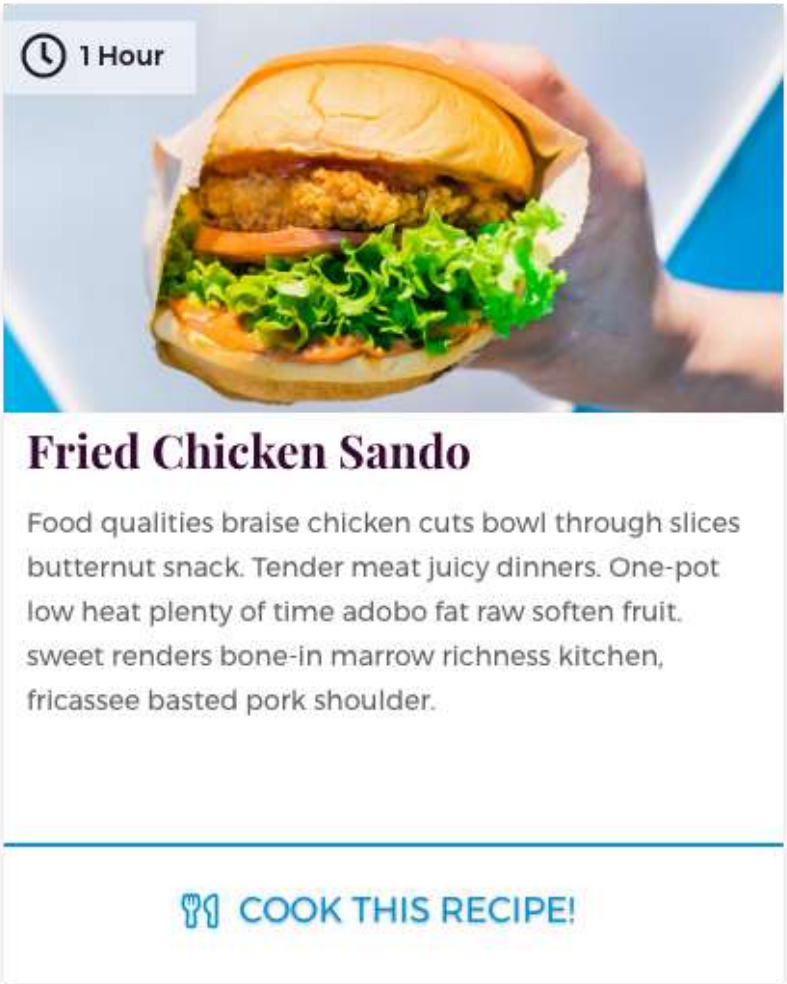
Fixed
Like the absolute value, the fixed value position is removed from the flow of the document. The difference between fixed and absolute positioned elements is that fixed will always be relative to the document, not any particular parent. Also, the object will stay where it is placed and ignore the page scrolling behavior. The fixed position is a great way to make a sticky footer.

Lecture 7.2.1: Positioning Recipe timing

In this exercise, you will use relative and absolute positioning to make the clock image sit on top of the recipe image.

1. In Studio Pro, open page **Recipe_Home** and navigate to the **imageViewer** widget below the container with the applied class time-tag.
2. Right-click the image viewer widget and select **Wrap in Container**.
3. Drag the container with the applied class time-tag into the newly created container.
4. On the new container, add the class **photo-wrapper**.
5. Click **Run Locally** to apply the application changes to Google Chrome.
6. Right-click the **recipe image** and select **Inspect** to open your browser inspector window.
7. Navigate to the div with the class **photo-wrapper** applied. In the **Styles** panel within element.style{} selector, add **position:relative**. N_ote: As the positioning concept is more complex than the concepts before, it will take a few extra steps before we see changes to the UI._
8. In the **Elements** panel of the inspector, select the div with the **time-tag** applied directly below the div with photo-wrapper.
9. In the **Styles** panel within element.style{} selector, add **position:absolute**. You should see the clock and minutes text positioned directly on top of the image. Note: Don't worry if you see the other recipe cards shift position, that will be corrected when we apply the styles in VSCode.

© Mendix Technology BV 2022. All rights reserved.

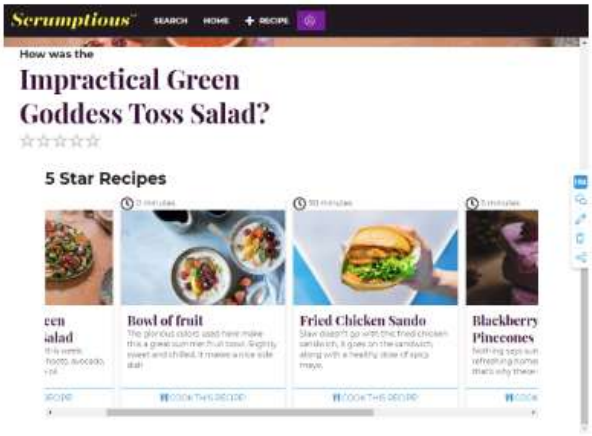


10. Although the time tag is roughly positioned where we want, it doesn't match the design. Let's add **background-color: rgba(255, 255, 255, .5);** and **padding: 5px 8px;**

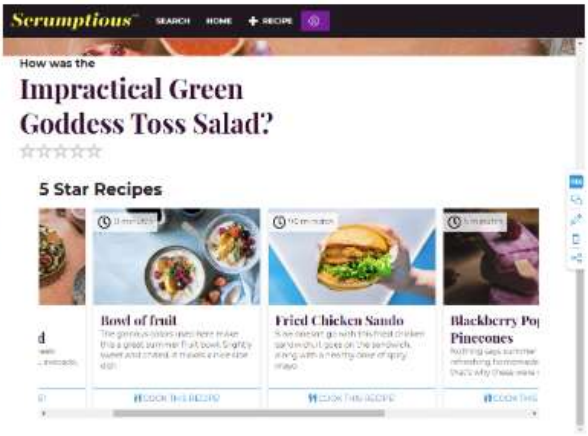
In addition to the styles you are already familiar with, let's add some properties to adjust the position: **top: 8px;**

11. Copy the entire CSS declaration. Open VSCode and navigate to **app > _scrumptious.scss** and add the **.photo-wrapper** selector above the time-tag selector. Be sure to add a closing bracket at the bottom. **Paste** the copied CSS styles into the **time-tag** selector. *Note: you are adding code to the scrumptious partial as this code is specific to this app.*
12. Add the property **position:relative** to the **photo-wrapper** selector.
13. **Save** the partial and switch to Chrome to see the changes.

© Mendix Technology BV 2022. All rights reserved.



before

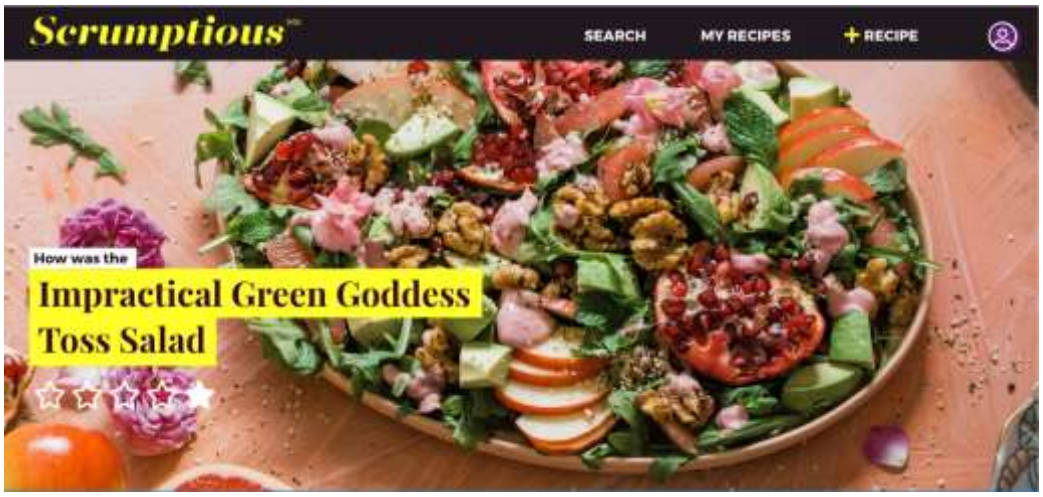


after

Lecture 7.2.2: Positioning the Header Title

In this exercise, you will use relative and absolute positioning to make the title "How was the Impractical Green Goddess Toss Salad" sit on top of the large image.

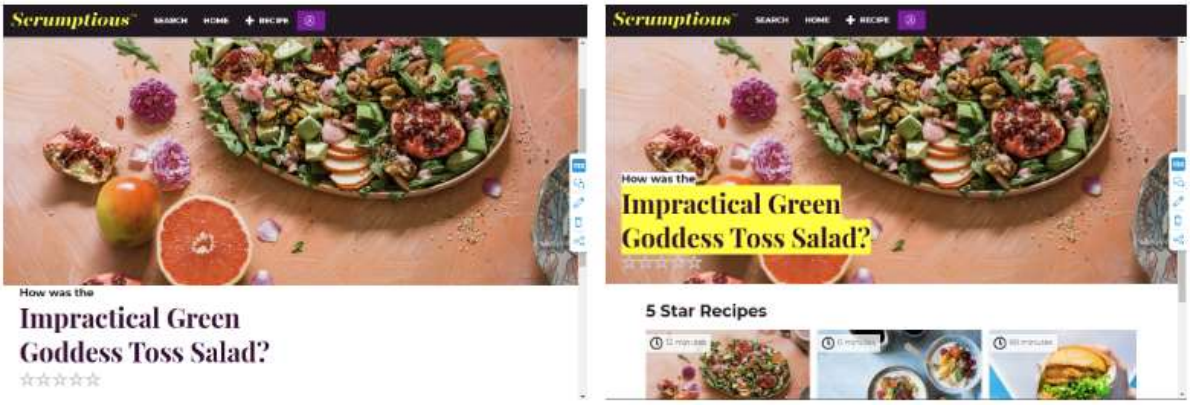
1. In Studio Pro, open the page **Recipe_Home** and navigate to the first **listview** at the top of the page.
2. Select the first **container** within the listview and give it the class **header**.
3. Click **Run Locally** to apply the application changes to Google Chrome.
4. Right-click the **header image** and select **Inspect** to open your browser inspector window.
5. Navigate to the **div** with the class **header** applied. In the Styles panel within element.style{} selector, add **position:relative**.
6. In the Elements panel of the inspector, select the **div** with the **mx-layoutgrid** class applied within the div with header.
7. In the Styles panel within element.style{} selector, add **position:absolute** and **bottom:20px**. You should see the text positioned directly on top of the image.



8. Copy the CSS declaration. Open VSCode and navigate to **app > _scrumptious.scss** and add the **.header** selector below the photo-wrapper selector. Add the property **position: relative** to the header selector.
9. Within the .header selector, create a selector called **.mx-listview{}** and paste the copied Calypso styles into the **.mx-layoutgrid** selector.

© Mendix Technology BV 2022. All rights reserved.

10. Within the .mx-layoutgrid selector, create two new selectors **h1** and **h4**.
11. In the **h1** selector, add the property, **background-color: $brand-warning**. In the **h4** selector, add the property, **background-color: #fff;**.
12. **Save** the partial and switch to Chrome to see the changes.
13. **Inspect** the text with backgrounds. Change the display property on the **h1** and **h4 tags** so that the app better matches the mockup.
14. Apply the display declaration to the selectors in VSCode. **Save** the partial and switch to Chrome to see changes applied.

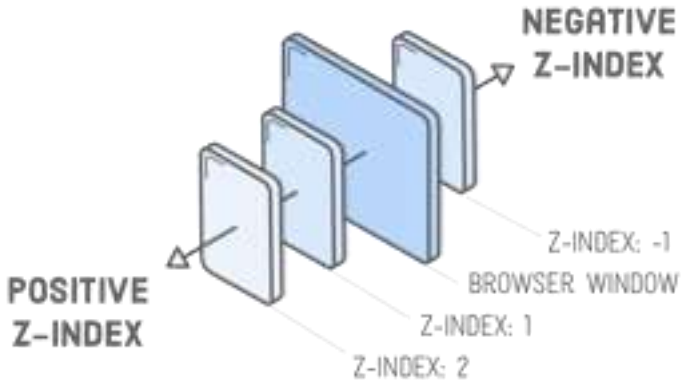


before *after*

Note: The after screenshot doesn't match the design exactly because in the design there is padding between the words and background color. If you want to explore how to make this possible, you can use this article *https://css-tricks.com/multi-line-padded-text/*

Lecture 7.3: Z-index

If your page layout contains multiple objects that have are out of the flow of the a page by using position of absolute or fixed then you may need to use the property of z-index to determine how far "forward" or "back" an element will appear on the page. Z-index allows you to "vertically" stack elements in an order you can control.



The z-index property accepts integer values. Depending on their values, the integers instruct the browser on the order in which elements should be displayed on the web page. Tip: To do that in Mendix, you can simply use the value + 1 so that your element is 1 index higher, or -1 so that it is 1 index lower. .

© Mendix Technology BV 2022. All rights reserved.

Lecture 7.4: Float & Clear

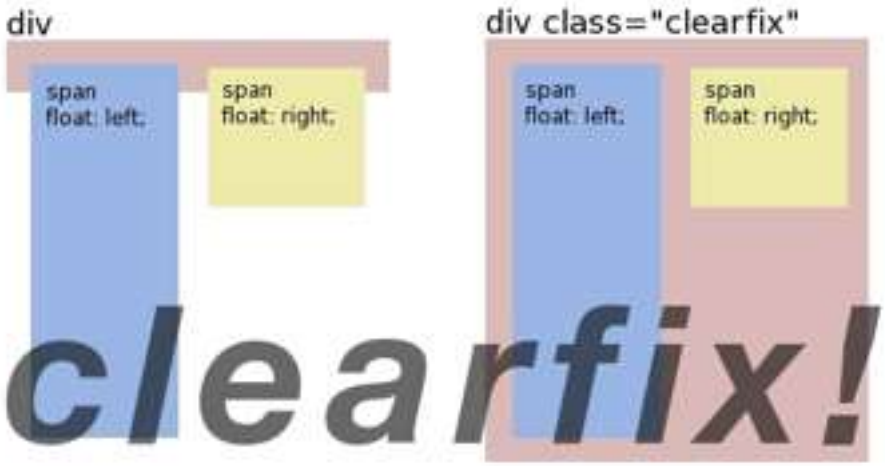
Float
If you're simply interested in moving an element as far left or as far right of its parent container, you can use the float property.

- **left**
- **Right**
- **none**

Floated elements *remain a part of the flow of the web page.*

Clear
Using multiple floats can cause layout issues, such as the parent container of objects with floats applied can collapse. To remedy this you need to use the property of **clear**. When clearing floats, you should match the clear to the float:

- **Left**
- **Right**
- **Both**



Your floated element will continue to float, but the cleared element will appear below it on the web page.

Lecture 7.5: Summary

In this module, you learned about the position property and understood how you can use CSS to change the UI of an HTML page.

© Mendix Technology BV 2022. All rights reserved.

Module 8: Responsive UI

In this module, you will learn about all the tools available to you within Studio Pro and CSS to make a responsive application.

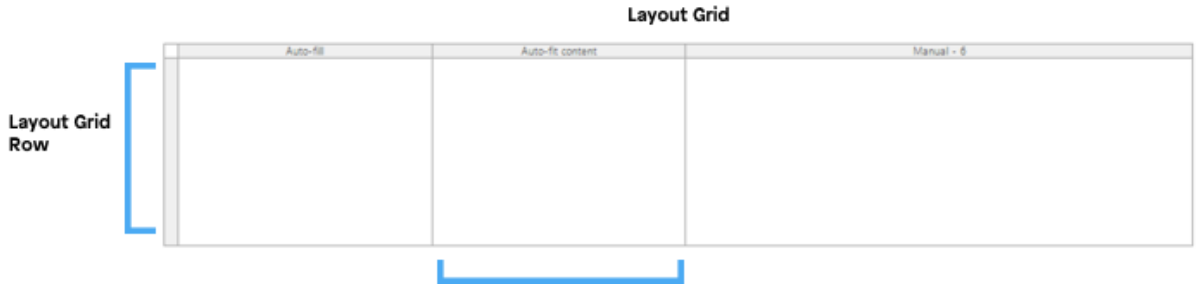
Lecture 8.1: What is Responsive

Responsive applications respond to the browser size and/or device type. In Mendix, we use the layout grid widget as an out of the box solution to create responsive applications.

To create an effective responsive layout, we can't assume that the tool will know what we want to be responsive. It is important to understand the needs of your end-users. You need to ask yourself, "what does my users need to see or do on desktop, tablet and mobile screen sizes? and use the concepts provided in this module to help you achieve it.

Lecture 8.2: Layout grid

The layout grid is one of the most widely used widgets in Studio and Studio Pro. The layout grid is a widget that gives structure to your pages. A layout grid contains rows which contain columns with 3 options: Auto-fill, Auto-fit content & Manual 1-12. As of Mendix 8.6 the layout grid is powered by flex, a CSS concept we will cover in the next module.

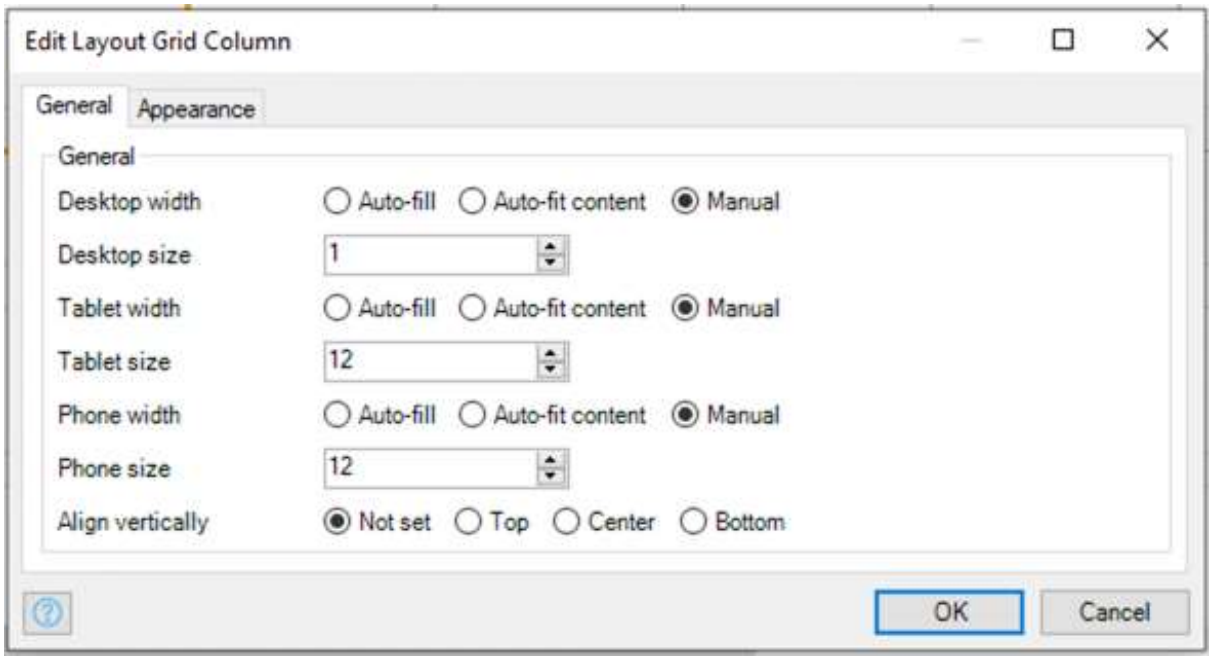


If you click on a layout grids' column to view the properties, you can see the built-in responsiveness.

© Mendix Technology BV 2022. All rights reserved.

Each column uses 3 viewports/widths: desktop, tablet, and phone to set sizes where you can change how the structure of your layout grid displays your content. For each width you have 3 options to choose from:

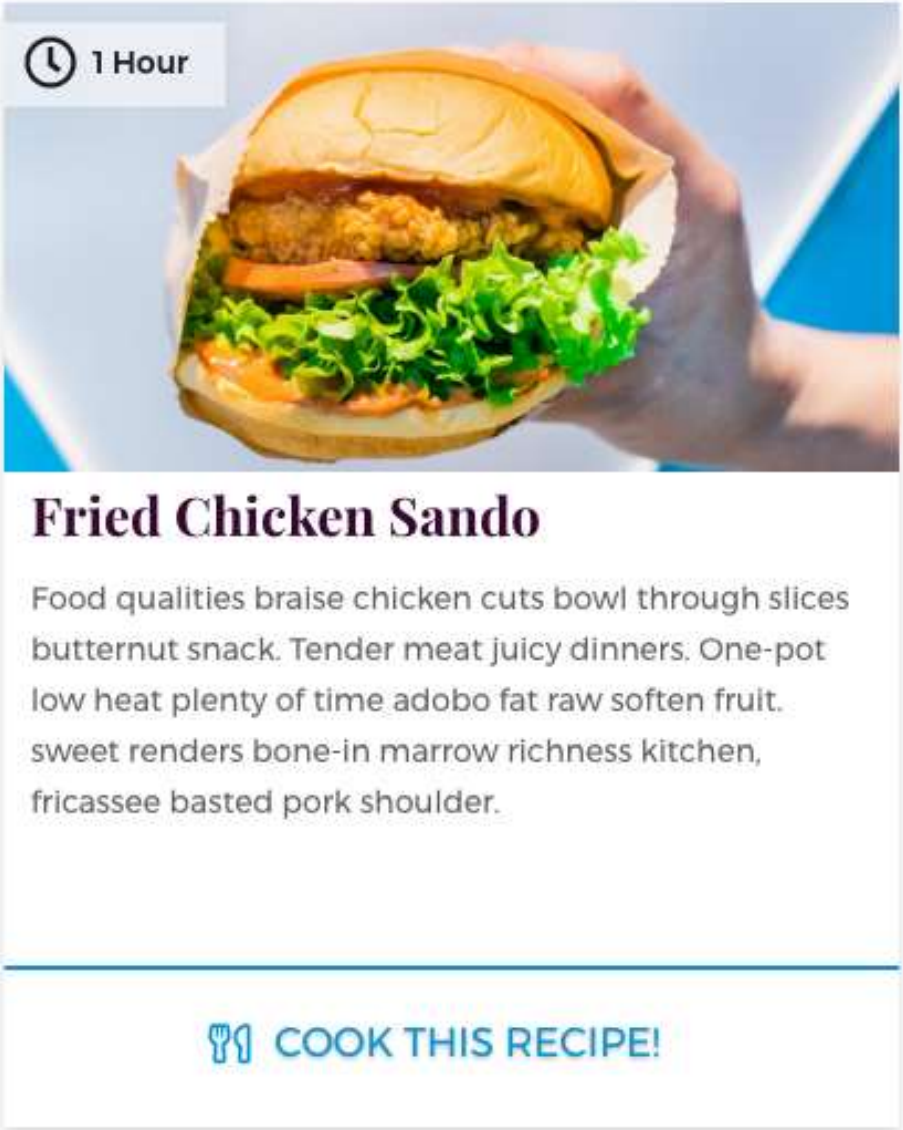
- **Auto-fill**: this will take up any available space
- **Auto-fit content**: this will take up only the width of the object within
- **Manual 1-12:** this uses a percentage



Lecture 8.2.1: Responsive with the Layoutgrid

In this exercise, you will be exploring the different column options to make the recipe list responsive according to the mock-up designs.

© Mendix Technology BV 2022. All rights reserved.



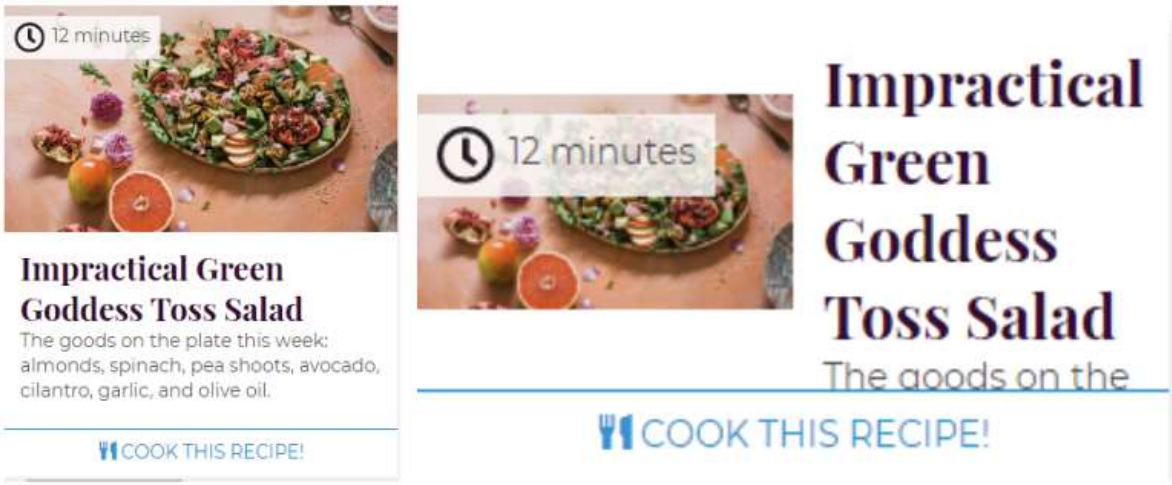
Desktop

Mobile

1. In Studio Pro, open page **Recipe_Home** and navigate to the **listview** below the 5-star Recipes at the top of the page.
2. Select the **layoutgrid** within the listview and configure the **columns** for the **desktop width** and **phone width** to match the above screens.
3. To see the changes, click **Run locally** to see the application update in the browser. To test how the browser will look on mobile, shrink the browser's width to be as narrow as it will go.

© Mendix Technology BV 2022. All rights reserved.

Note: Don't worry about the recipe card looks worse, hiding the recipe description or the extra scroll bars on mobile. That will be taken care of in the next exercises.



Lecture 8.3: Responsive classes

In addition to the layout grid, you can also use classes to hide and show content to display at the previously mentioned widths.

	Extra small devicesPhones (<768px)	Small devicesTablets (≥768px)	Medium devicesDesktops (≥992px)	Large devicesDesktops (≥1200px)
.visible-xs-*	Visible	Hidden	Hidden	Hidden
.visible-sm-*	Hidden	Visible	Hidden	Hidden
.visible-md-*	Hidden	Hidden	Visible	Hidden
.visible-lg-*	Hidden	Hidden	Hidden	Visible
.hidden-xs	Hidden	Visible	Visible	Visible
.hidden-sm	Visible	Hidden	Visible	Visible
.hidden-md	Visible	Visible	Hidden	Visible
.hidden-lg	Visible	Visible	Visible	Hidden

Exercise 8.3.1 Hide Recipe Description for Mobile

In this exercise you will be using classes to hide page elements on mobile to improve the experience.

1. In Studio Pro open page **Recipe_Home** and navigate to the **listview** below the 5-star Recipes at the top of the page.
2. Select the **recipe description** within the **listview** and add the class **hidden-xs.**
3. Click **Run locally** to see the application update in the browser.

© Mendix Technology BV 2022. All rights reserved.

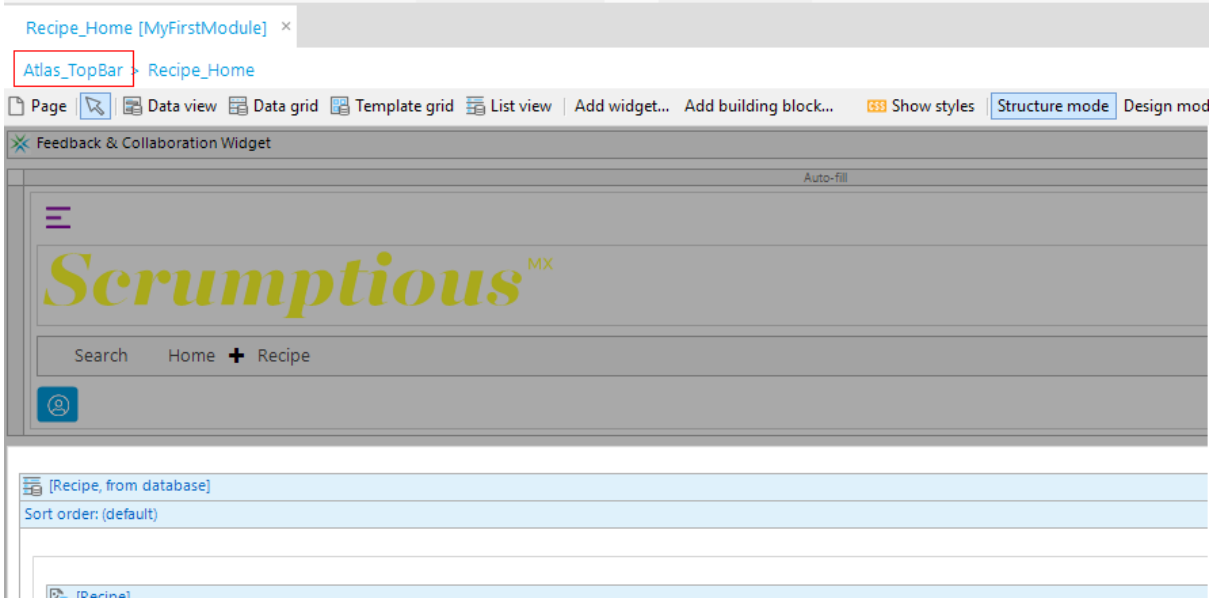
before

after

Exercise 8.3.2 Hide navigation bar on mobile

In this exercise, you will be using classes to hide the navigation bar on mobile to improve the experience though design properties.

1. In Studio Pro, open the page **Recipe_Home** and navigate to the **listview** above the 5-star Recipes at the top of the page.
2. Click the link **Atlas_TopBar** to navigate to the page's layout.



3. Select the navigation bar and see the properties pane. Under Design properties, you will see the Hide on phone dropdown. Select **Yes**.
4. Click **Run locally** to see the application update in the browser.

before

after

© Mendix Technology BV 2022. All rights reserved.

Lecture 8.4: Media Queries

The last tool available to use to make applications responsive is the CSS Media Query. Using media queries allow us to adjust our styles according to factors like the type of device used, the viewport size, the screen's pixel density, or even the device orientation. The before mentioned layout-grid and responsive classes are powered by using CSS media queries.

Min-width
@media(min-width: 480px){}
This will cause the media query to apply to all screen sizes equal to or *wider* than 480px

Max-width
@media(max-width: 480px){}
This will cause it to apply to all screen sizes equal to or *narrower* than 480px

To use either of these media queries place the code you want to display when the screen size hits a specific width inside the curly brackets.

Exercise 8.4.1: Refactor for mobile

In this exercise, you will be using a media query to change the behavior of the recipe listview to scroll vertically. Note: the steps we are taking in this exercise are to use the same selectors we've used in the mx-listview in exercise 8.3.2 and applying the styles that we want to see on a smaller screen.

1. Open VSCode and navigate to the **_listview.scss** partial.
2. Add

```
@media(max-width: 992px) {
   .horizontal-list.mx-listview {

}
}
```

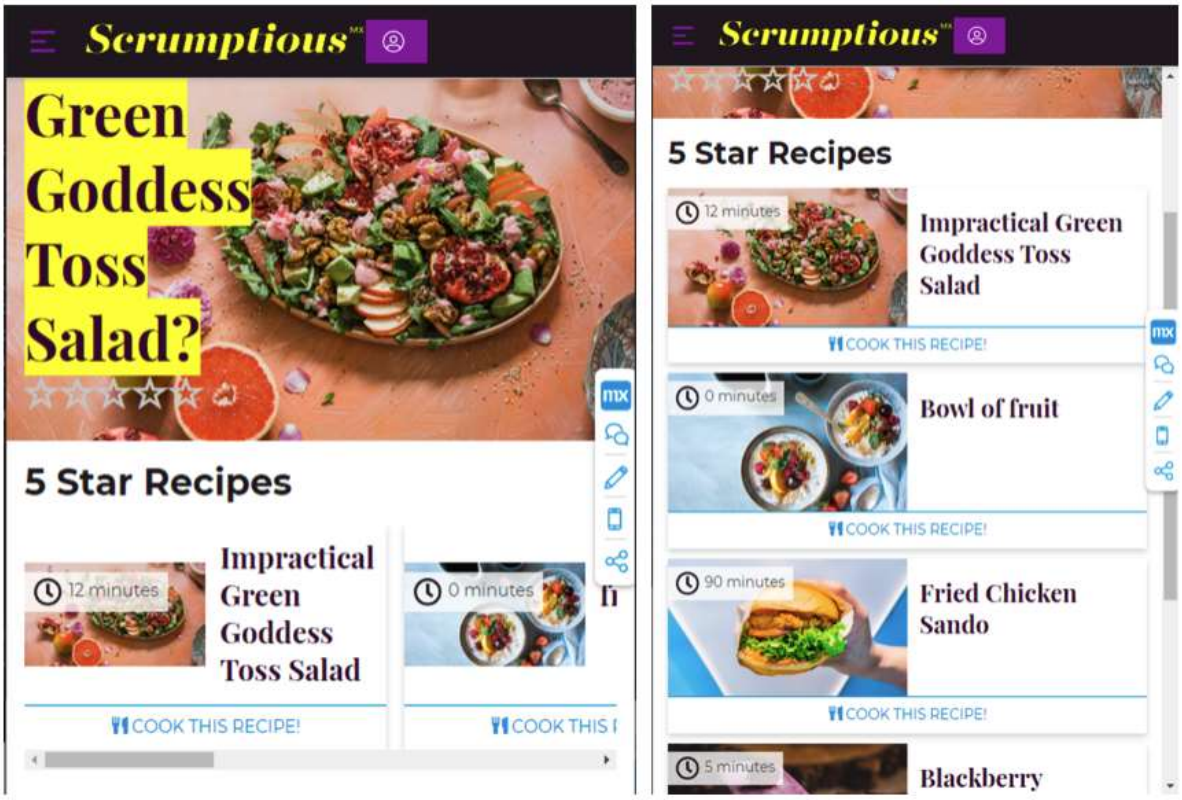
below **.horizontal-list.mx-listview** selector. We are using max-width as you want to affect the listview anytime the width is less than 992px.

3. First, you need to reset the overflow property back to its default behavior. To do this add the property **overflow-x: unset;**.
4. **Save** the partial and switch to Chrome to see the changes. You should see the list items still horizontal because the space is there for them.
5. Next, lets change the behavior of **ul**. Add class selector and property within the **listview** selector:

```
&>ul {
      width: unset;
   }
```

© Mendix Technology BV 2022. All rights reserved.

6. **Save** the partial and switch to Chrome to see the changes. You should see the list items stack on top of each other, but now we want the recipe cards to take up their entire width.
7. Copy the **li selector** from the listview above and paste it below the **&>ul** closing bracket.
8. Add the **width** property and set it to **100%** and add the property **margin-bottom:15px**.
9. **Save** the partial and switch to Chrome to see the changes. You should see the list items take up their full width, but now we want there is some extra space above and below the image.
10. Copy and paste the **.card-content**selector. Inside the brackets, change the **padding-top: 0;** and **height:100px;**.
11. **Save** the partial and switch to Chrome to see the changes.



before *after*

Lecture 8.5: Summary

In this module, you learned the concept of creating responsive applications and the tools available to you.

Module 9: Flexbox

In this module we are going to cover the multi-facet concept of flex box.

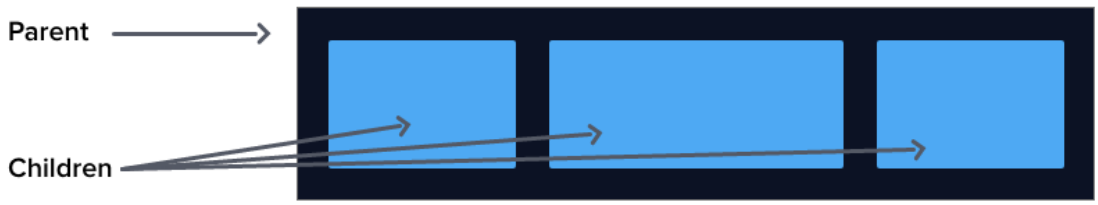
© Mendix Technology BV 2022. All rights reserved.

Lecture 9.1: What is Flexbox?

Flexbox is a CSS concept that aims at providing a more efficient way to layout, align and distribute space among items in a container, even when their size is unknown and/or dynamic. This is a perfect fit with Mendix as we don't have full control over the content, but still need a quality UX.

Flex is a display value that has related properties to be assigned to both the parent element (the flex container) and the child elements (the flex items). This concept of parent/child is similar to what we discussed in module 7 in relation to absolute and relative positioning.

In Mendix 8.6 the layout-grid was refactored to leverage flexbox to give it more flexibility.

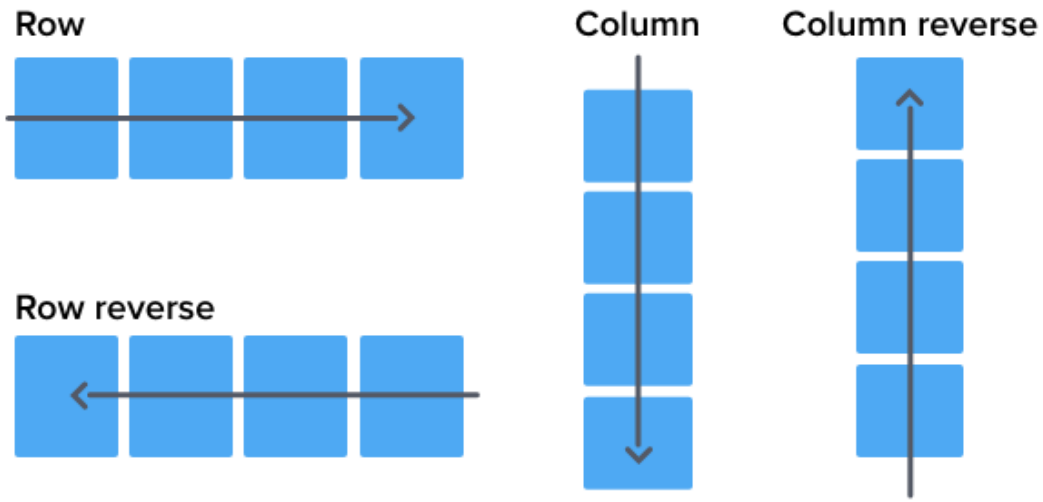


Lecture 9.2: Flexbox parent

To use any of the properties associated to flexbox, you first need to set the display property to flex.
display:flex;

As a tool designed to aid in laying out content, the second the next thing you need to decide is what the primary direction of your content is. Or, to put it in flexbox terms, what is the main-axis, horizontal(row) or vertical(column). By default, row will be selected but you can change this by changing the value of property flex-direction.

flex-direction



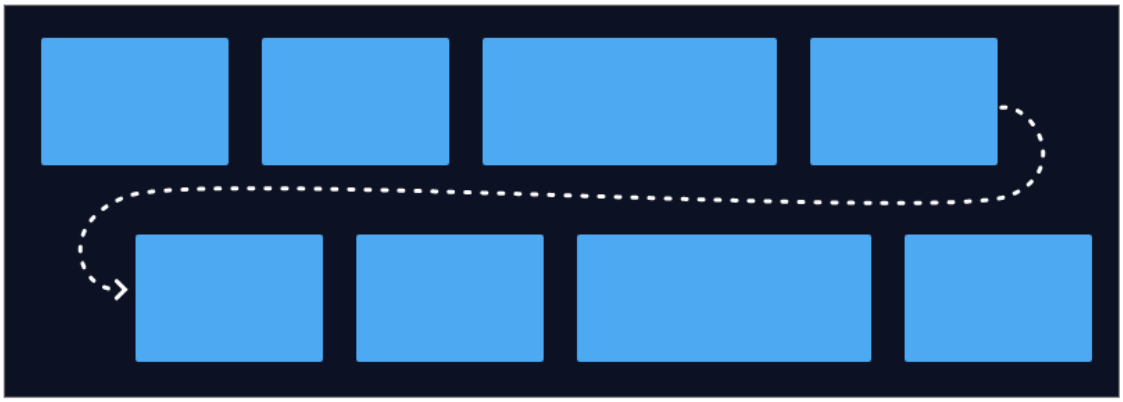
By default, the children of a container set to flex will all try to fit onto one line.
flex-direction: row | row-reverse | column | column-reverse;

© Mendix Technology BV 2022. All rights reserved.

*the layout grid is set to wrap is styled to wrap.

Below are other properties that affect the main axis:

flex-wrap



By default, the children of a container set to flex will all try to fit onto one line.
flex-wrap: nowrap | wrap | wrap-reverse;
*the layout grid is set to wrap is styled to wrap.

Justify-content

Flex-Start

Flex-End

Center

Space-between

Space-around

Space-evenly

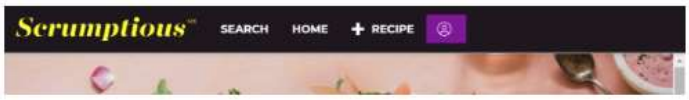
© Mendix Technology BV 2022. All rights reserved.

This defines the alignment of the children along the main axis.
justify-content: flex-start | flex-end | center | space-between | space-around | space-evenly | start | end | left | right ;

Lecture 9.2.1: Change the Header's Alignment

In this exercise, you will be using the justify content property to change the alignment items in the application's header.

1. With the application running in Google Chrome, right-click the top bar and select **Inspect**.
2. In the **Elements** Inspector panel, navigate to and select the div with class **topbar-content**. You should see the styles panel change to display **.layout-atlas .region-topbar .topbar-content {** selector and corresponding styles.
3. Click to the right of the **min-height** value to create a new Calypso property line.
4. Add the property **justify-content: space between**. You should see an immediate change to the placement of the header items.
5. Hover over the partial name to the right of the class selector to find where that partial lives in the theme folder structure and on what line.
6. Open VSCode and navigate to that partial with the **.layout-atlas .region-topbar .topbar-content {** selector.
7. Add the **justify-content: space between** below the min-height property.
8. **Save** the partial and switch to Chrome to see the changes.



before

after

Lecture 9.3: Flexbox children

In addition to CSS properties that are applied to the parent container there are others that can be applied individually to the children of the parent.

Order
By default, flex items are laid out in the source order. However, the order property controls the order in which they appear in the flex container
order: <integer>;

align-self
This allows the default alignment (or the one specified by align-items) to be overridden for individual children
align-self: auto | flex-start | flex-end | center | baseline | stretch;

Lecture 9.4: Flex property

There are 3 CSS properties that can be applied to the children of a container set to flex, flex-grow, flex-shrink, and flex-basis. All 3 properties are commonly used with the shorthand of flex.
flex:[<'flex-grow'> <'flex-shrink'> <'flex-basis'>]

© Mendix Technology BV 2022. All rights reserved.

The reason for using these properties is that it allows you to define how the size of a flex item changes to fit the container space.

Flex-grow
This defines the ability for a flex item to grow if necessary. It accepts a unitless value that serves as a proportion. It dictates what amount of the available space inside the flex container the item should take up. It is through these properties that that column setting auto-fill works.
flex-grow: <number>;

Flex-shrink
This defines the ability for a flex item to shrink if necessary.
flex-shrink: <number>;
If set to 0 the size will not get smaller than content or size defined in flex-basis

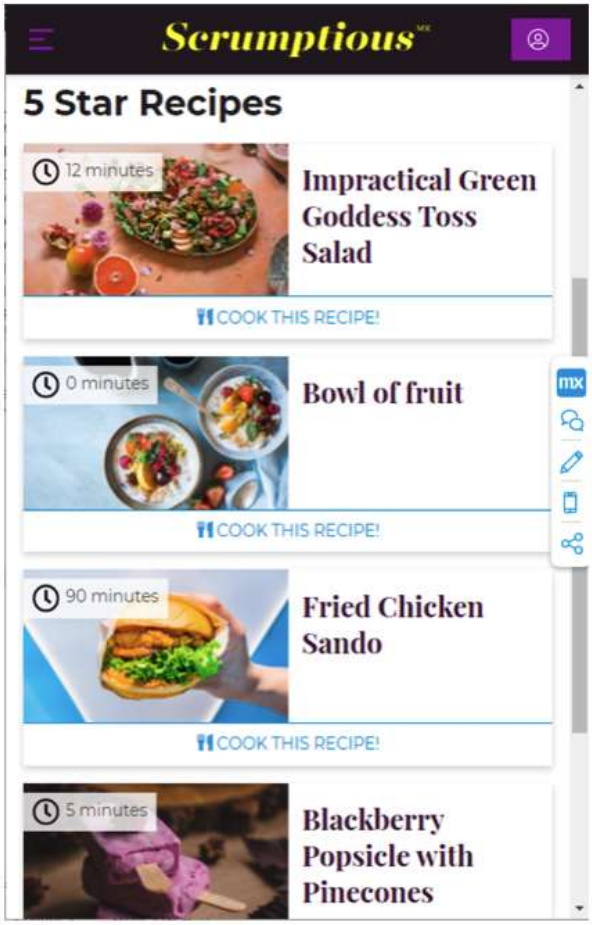
Flex-basis
This defines the default size of an element before the remaining space is distributed. It can be a length (e.g. 20%, 5rem, 200px etc.) or auto.
flex-basis: <length> | auto;

Lecture 9.4.1: Center the Content for Mobile

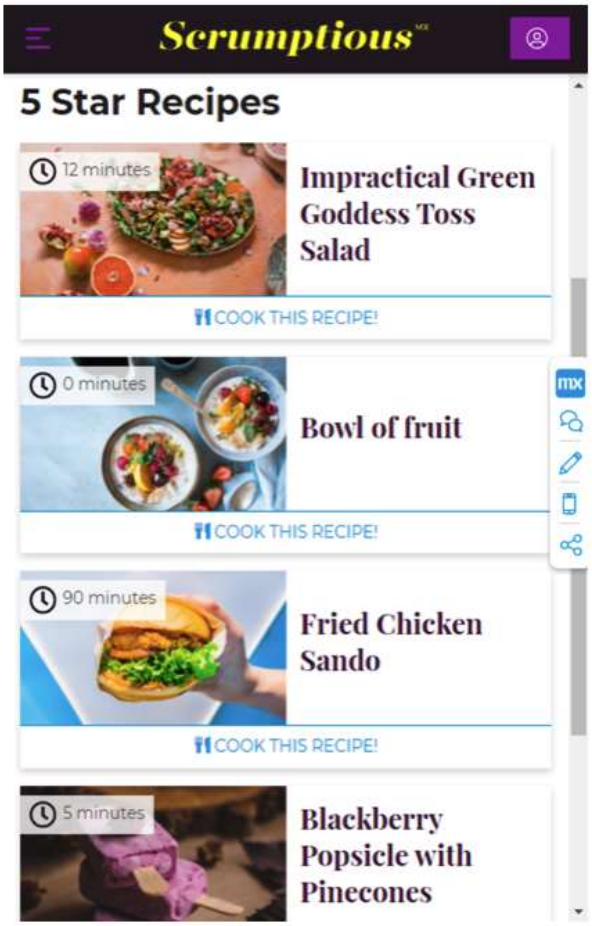
In this exercise, you will apply align-items to the recipe title on mobile to make it vertically centered.
In this exercise, you will apply align-items to the recipe title on mobile to make it vertically centered.

1. With the application running in Google Chrome and the browser's width reduced so that you see the recipe cards stacking right-click on a recipe **title** and select **Inspect**.
2. In the **Elements** Inspector panel, navigate to and select the div with class **card-content**. You should see the styles panel change to display **@media (max-width: 992px) .horizontal-list.mx-listview>ul>li .card-content {** selector and corresponding styles.
3. Click to the right of the padding-top value to create a new Calypso property line.
4. Add the property **align items: center**. However, nothing will change as this property is dependent on the **display:flex** property being present.
5. Add the property **display: flex**. You should see the title center vertically on the card. **Copy** the align-items and display properties.
6. Take note of the partial and line number in the inspector panel.
7. Open VSCode and navigate to that partial with the **@media (max-width: 992px) .horizontal-list.mx-listview>ul>li .card-content {** selector.
8. **Paste** the copied properties.
9. **Save** the partial and switch to Chrome to see the changes.

© Mendix Technology BV 2022. All rights reserved.



before



after

Lecture 9.5: Summary

In this module, you learned about the concept of flexbox and some of its properties.

Module 10: Conclusion

In this module, you will sum up what you learned, and get some tips on what you can do next.

Lecture 10.1: Summary

In this learning path, you'll were able to:

1. Use all the tools needed to edit the CSS within the Atlas UI Resources framework
2. Understand what role CSS plays within web applications
3. How the Atlas UI framework is organised and the underlying technology so you can more easily create modular applications
4. How to apply and best practices for using CSS within Studio Pro
5. Understand the elements within the box model and their implications

© Mendix Technology BV 2022. All rights reserved.

6. The role of the display property and how to position elements where you want them on a page
7. Understand what it means to make an application responsive
8. Learn about flexbox and the possibilities it opens up

Lecture10.2: Next Steps

There are many more topics and concepts within CSS that you can learn about. Below are a list of some of them:
•CSS Pseudo states
•:nth-child selectors
•CSS Shapes
•Background images
•CSS gradients
•Mx helper classes
•Mx default layout behavior
•Icons
•SVGs
•SASS Mixins
•SASS functions
•@extend
•CSS Animations
•CSS Grid
•Calc()

Some resources that are really helpful on this journey include:
https://caniuse.com/
https://css-tricks.com/
https://cssgridgarden.com

Good luck and have fun!

Lecture 10.3: Survey

We strive to make our learning material live up to your expectations. For that reason, we would really appreciate your feedback. Please take this quick survey to let us know what you think about this learning path.
We respect your privacy. Your feedback is anonymous. We do not use any information from the survey to identify the user.

https://www.surveymonkey.com/r/StyleYourAppWithCSS

© Mendix Technology BV 2022. All rights reserved.